AeroSense

Fire Risk Monitoring System – Project Report

AeroSense- Process Report Semester Project 4

Group Y1

Students
Felipe Figueiredo (355463)
Guillermo Sánchez (355442)
Waqar Ahmed Khan (355425)
Eliza Manciu (204590)
Piotr Gała (355451)
Matteo Saccucci (355400)
Matteo De Filippis (355438)
Soma Nagy (355465)

Supervisors
Marketa Tranberg
Joseph Chukwudi Okika
Kasper Knop Rasmussen
Erland Ketil Larsen

Character Count: 143116
Word Count: 22066

Software Technology Engineering 4th Semester

May 2026
Process Report VIA University College

1 Abstract

This report presents the analysis, design, implementation, DevOps workflow, and testing of AeroSense, a fire risk and air-quality monitoring system for indoor rooms. The system uses IoT devices to collect sensor data such as temperature, humidity, CO_2, TVOC, and smoke or air-quality indicators. Based on this data, AeroSense is intended to show current and historical room conditions, detect or predict possible fire risk, distinguish between normal conditions, cooking smoke, steam or false alarms, and unhealthy air quality, and provide warnings, notifications, recommendations, and alarms when needed.

The project was developed as a full system consisting of IoT, Backend, Machine Learning, and Frontend parts. The analysis phase defined the main actors, user stories, functional and non-functional requirements, use cases, and domain model. The design and implementation sections describe how the different subsystems work together, including sensor data collection, MQTT communication, backend logic, machine learning predictions, dashboard functionality, CI/CD, and testing. The report also discusses the results, limitations, and possible future improvements of the system.

2 Introduction

Author: Rebeca Proskovcova

AeroSense is a fire risk and air quality monitoring system designed for rooms and buildings where early awareness of environmental changes is important. The system addresses the problem of unreliable or incomplete monitoring in shared indoor spaces, especially in situations where traditional alarms may be triggered by everyday activities such as cooking, steam, or other non-dangerous events. Repeated false alarms can create unnecessary disruption and costs, but they can also reduce how seriously people react when a real emergency occurs.

The main idea of AeroSense is to collect environmental data from rooms and use it to support smarter monitoring and decision-making. Instead of relying only on a simple alarm trigger, the system observes values such as CO2, temperature, and humidity, and uses these readings to give a clearer picture of the current room conditions. This makes it possible to monitor both potential fire risks and general air quality in a more informative way.

The project is built around four technical areas: IoT, Backend, Machine Learning, and Frontend. The IoT part is responsible for collecting sensor readings from the physical environment. The Backend handles data storage, communication, authentication, and the main system logic. The Machine Learning part supports analysis and prediction based on collected data, helping the system identify patterns that may indicate unsafe or unusual conditions. The Frontend provides a user interface where readings, room status, historical data, and alerts can be viewed in a clear and accessible way.

The scope of this project is to design and implement a prototype of AeroSense that demonstrates how sensor data, cloud-based services, machine learning, and a web dashboard can work together in one connected system. The system is not intended to replace certified fire alarm infrastructure, but rather to act as a monitoring and decision-support solution. This report covers the analysis, design, implementation, testing, and evaluation of the system, including the work done across the IoT, Backend, Machine Learning, and Frontend parts of the project.

2.2 Report Purpose

The purpose of this report is to document the development of the AeroSense project from analysis to final evaluation. It describes the system requirements, design decisions, implementation process, testing activities, CI/CD setup, achieved results, and possible future improvements. The report also explains how the different technical parts of the system were developed and integrated into one working prototype.

2.3 Scope of This Report

This report focuses on the technical development and evaluation of the AeroSense system. A detailed explanation of the project background, problem domain, problem statement, delimitations, and overall project scope is provided in the separate Project Description document. Therefore, this report only summarizes those areas briefly and mainly focuses on how the system was analyzed, designed, implemented, tested, and evaluated.

2.4 Report Structure

The report begins with a common introduction to the project, including the general context, purpose, scope, and overall requirements. The analysis section presents the shared understanding of the system before the report moves into the design section. From the design section onward, the report is divided into subteam-specific parts covering IoT, Backend, Machine Learning, and Frontend. Each subteam describes its own design choices, implementation, testing, and relevant technical work. Later sections are merged again to present the complete system integration, CI/CD process, results, discussion, future work, and conclusion.

3. Main Section

3.1 Analysis

Author: Eliza Bianca Manciu

3.1.1 Analysis Overview

The analysis phase was used to create a shared understanding of what the AeroSense system should do before the project was divided into IoT, Backend, Machine Learning, and Frontend work. Since the system has several technical parts, it was important to first describe it as one coherent solution.

This analysis section therefore presents the shared foundation for the whole AeroSense system. The detailed technical design of each subsystem is described later in the report after the common design overview.

3.1.2 System Actors

This section describes the main actors who interact with the AeroSense system through the dashboard, notifications, alerts, or administration features. The main human actors are Resident, Building Administrator, and System Administrator.

Resident

A Resident is a person who lives in, or uses, a room monitored by AeroSense. The Resident can view current and historical room conditions, receive warnings, notifications, and recommendations, and react to possible fire risk or unhealthy air quality.

Building Administrator

A Building Administrator monitors several rooms or buildings from a broader overview. This actor needs access to room conditions, alerts, fire status, historical data, and rooms with higher predicted risk. The role is important for reacting to possible fire risks, air-quality problems, or device-related issues.

System Administrator

A System Administrator manages the technical side of the system. This includes users, roles, access rights, connected IoT devices, device status, device configuration, and system logs. This actor helps ensure that the system is correctly configured and that only registered devices are accepted.

Supporting System Components

AeroSense also depends on technical components such as the IoT device, Machine Learning model, and Backend. These are not treated as human actors in the use case model, but they are important system parts. The IoT device collects sensor data and can activate the buzzer alarm, the ML model supports classification and prediction, and the Backend handles logic, data storage, and communication between the subsystems.

3.1.3 User Stories

User stories were created to describe the AeroSense functionality from the user's perspective. They helped clarify what each actor needs from the system and were later used as a foundation for use cases, requirements, and design decisions.

In total, 23 user stories were created. The complete list of user stories can be found in **Appendix B.1.**

3.1.4 Functional Requirements

Functional requirements describe what the AeroSense system must be able to do. They were created based on the user stories and were used as a foundation for use cases, design, implementation, and testing.

The functional requirements for AeroSense are:

1. **FR1:** The system shall allow a resident to view the current room conditions, including temperature, humidity, CO_2, and TVOC levels.
2. **FR2:** The system shall allow a resident to view past room condition data.
3. **FR3:** The system shall detect or predict a possible fire based on sensor data and warn the resident through a notification and a sound alarm.
4. **FR4:** The system shall distinguish between cooking-related smoke, possible fire, and false-alarm steam smoke.
5. **FR5:** The system shall notify a resident when the air quality becomes unhealthy.
6. **FR6:** The system shall provide a resident with recommendations for improving air quality, including recommending ventilation when unhealthy cooking smoke is detected.
7. **FR7:** The system shall predict when the air quality is likely to become unhealthy.
8. **FR8:** The system shall allow a resident to mute or dismiss non-critical alerts and sound alarms when the situation is resolved or no danger is detected.
9. **FR9:** The system shall allow a building administrator to view the overall room conditions and fire alarm status of multiple rooms on one dashboard.
10. **FR10:** The system shall notify a building administrator when a possible fire is detected and show in which room it occurred.
11. **FR11:** The system shall show a building administrator which room currently has the highest predicted immediate fire risk.
12. **FR12:** The system shall allow a building administrator to review historical room data and past alerts.
13. **FR13:** The system shall identify rooms or areas with higher predicted fire risk based on historical data and recurring sensor patterns.
14. **FR14:** The system shall allow a building administrator to dismiss notifications and silence or reset alarms after the situation has been checked and resolved.
15. **FR15:** The system shall allow a system administrator to manage user accounts, roles, and access rights.
16. **FR16:** The system shall allow a system administrator to add, configure, and connect IoT devices from multiple rooms to the system.
17. **FR17:** The system shall show the system administrator whether devices are online or offline.
18. **FR18:** The system shall allow a system administrator to reset or temporarily disable malfunctioning devices.
19. **FR19:** The system shall allow a system administrator to view system logs.

These requirements cover the main expected behavior of AeroSense, including room monitoring, fire and air-quality risk handling, recommendations, administration, and device management.

3.1.5 Non-functional Requirements

Non-functional requirements describe how the AeroSense system should behave. They focus on quality expectations such as usability, reliability, performance, operations, and security.

The non-functional requirements for AeroSense are:

1. **NFR1 – Usability:** The system shall present room conditions, alerts, and recommendations in a clear and easy-to-understand way for all user roles.
2. **NFR2 – Usability:** The system shall use a simple dashboard layout so that residents and building administrators can easily find the most important information.
3. **NFR3 – Reliability:** The system shall continue monitoring room conditions automatically every 5 seconds without requiring constant manual input.
4. **NFR4 – Reliability:** The system shall log important alerts and system events so that problems can be reviewed later.
5. **NFR5 – Reliability:** If a device loses connection, the system shall show this clearly to the system administrator within 30 seconds.
6. **NFR6 – Performance:** The system shall update current sensor data and alerts within 5 seconds under normal conditions.
7. **NFR7 – Performance:** The system shall show dashboard pages and room condition views within 2 seconds under normal use.
8. **NFR8 – Performance:** The system shall send a fire warning within 5 seconds after the system classifies the room condition as a possible fire.
9. **NFR9 – Performance:** The system shall send a fire risk warning within 5 seconds after the system predicts fire risk.
10. **NFR10 – Operations:** The system shall keep historical sensor data, alerts, and logs available for later review.
11. **NFR11 – Security:** Each IoT device should have its own ID or key, and the system should only accept data from registered devices.

These requirements are important because AeroSense works with safety-related warnings and room monitoring. The system should therefore present information clearly, react fast enough, log important events, and handle device connections reliably.

3.1.6 Use Case Diagram

The use case diagram gives an overview of the main interactions between the AeroSense system and its actors. It shows the most important system functionalities from the user's point of view, without describing the internal technical implementation.

The diagram includes three main actors: **Resident**, **Building Administrator**, and **System Administrator**. The Resident can view room conditions, receive air-quality advice and predictions, classify room conditions, and receive fire warnings. The Building Administrator can monitor the building safety status and receive information about fire warnings and predictions across rooms. The System Administrator can manage users, access, IoT devices, and system status.

The use case diagram was kept at a high level to show the main system goals clearly. Since the diagram is inserted in a smaller size in the main report, a larger and more readable version is included in **Appendix B.2**.

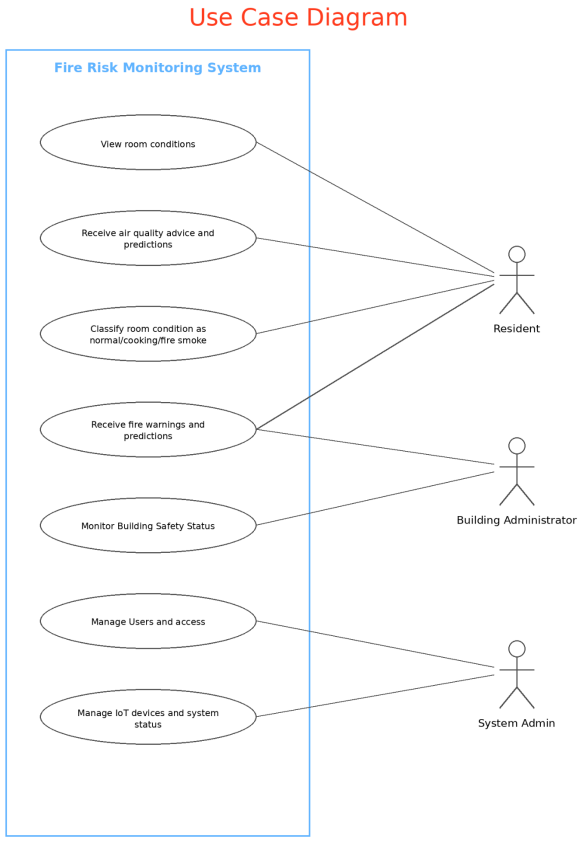

Figure 1 – Use Case Diagram for AeroSense

3.1.7 Use Case Descriptions

Use case descriptions were created to describe selected system interactions in more detail than the use case diagram. While the use case diagram gives a high-level overview of what each actor can do, the use case descriptions explain the flow of events for specific use cases.

Each use case description includes the main actor, preconditions, postconditions, main success scenario, and possible alternative flows. This helped clarify how the system should behave in normal situations and what should happen if something changes, such as missing data, unhealthy air quality, or a resolved alarm.

The use case descriptions were also used as a bridge between the user stories, requirements, and later design decisions. They helped make the expected system behavior more concrete before moving into implementation. The full use case descriptions are included in **Appendix B.3.**

3.1.8 Activity Diagram

An activity diagram was created for the use case **Receive Fire Warning and Fire Prediction**. This flow was chosen because it shows one of the most important parts of AeroSense: how the system goes from sensor data collection to fire risk evaluation, notification, buzzer activation, and user response.

The diagram clarifies how the system reacts when room conditions are normal, when unhealthy air quality is detected, and when a possible fire or high fire risk is identified. The full activity diagram is included in **Appendix B.4.**

3.1.9 System Sequence Diagram

A system sequence diagram was created for the use case **View Room Conditions**. This use case was chosen because it shows a clear interaction between the Resident and the system.

The SSD shows the system as one black box and only focuses on the messages between the actor and the system. It does not include internal implementation details such as frontend components, backend services, database calls, IoT logic, or ML processing.

The diagram shows the main success scenario where the Resident opens the dashboard, selects a room, views the current room conditions, and requests historical room data. The full system sequence diagram is included in **Appendix B.5.**

3.1.10 Domain Model

The domain model was created to give a conceptual overview of the most important concepts in the AeroSense system and how they are related. It is not meant to show database tables, implementation classes, primary keys, or foreign keys. Instead, it shows the problem domain before the system is described in detailed design.

During the analysis phase, the first version of the model was too close to a database or class diagram, so it was redesigned to be more conceptual. The final model is divided into the main areas **Users, Location, Monitoring, Analysis, Response, and Administration**. This helped separate the different parts of the system while still showing how they connect.

The model shows how users such as Residents, Building Administrators, and System Administrators are connected to monitored buildings and rooms. Rooms are equipped with IoT devices, which produce sensor readings such as temperature, humidity, CO_2, TVOC, and smoke level. These readings are then used to assess room conditions and create predictions.

The model also shows how the system responds to analysis results. Room conditions and predictions may trigger alerts, notifications, alarms, or recommendations. System logs are used to document important events such as alerts, device events, and system problems.

Overall, the domain model gives a shared understanding of the AeroSense problem domain and was used as a foundation for the later design work. A larger and more readable version of the domain model is included in **Appendix B.6**, since the version in the main report is reduced in size.

3.1.11 Current Domain Model Figure

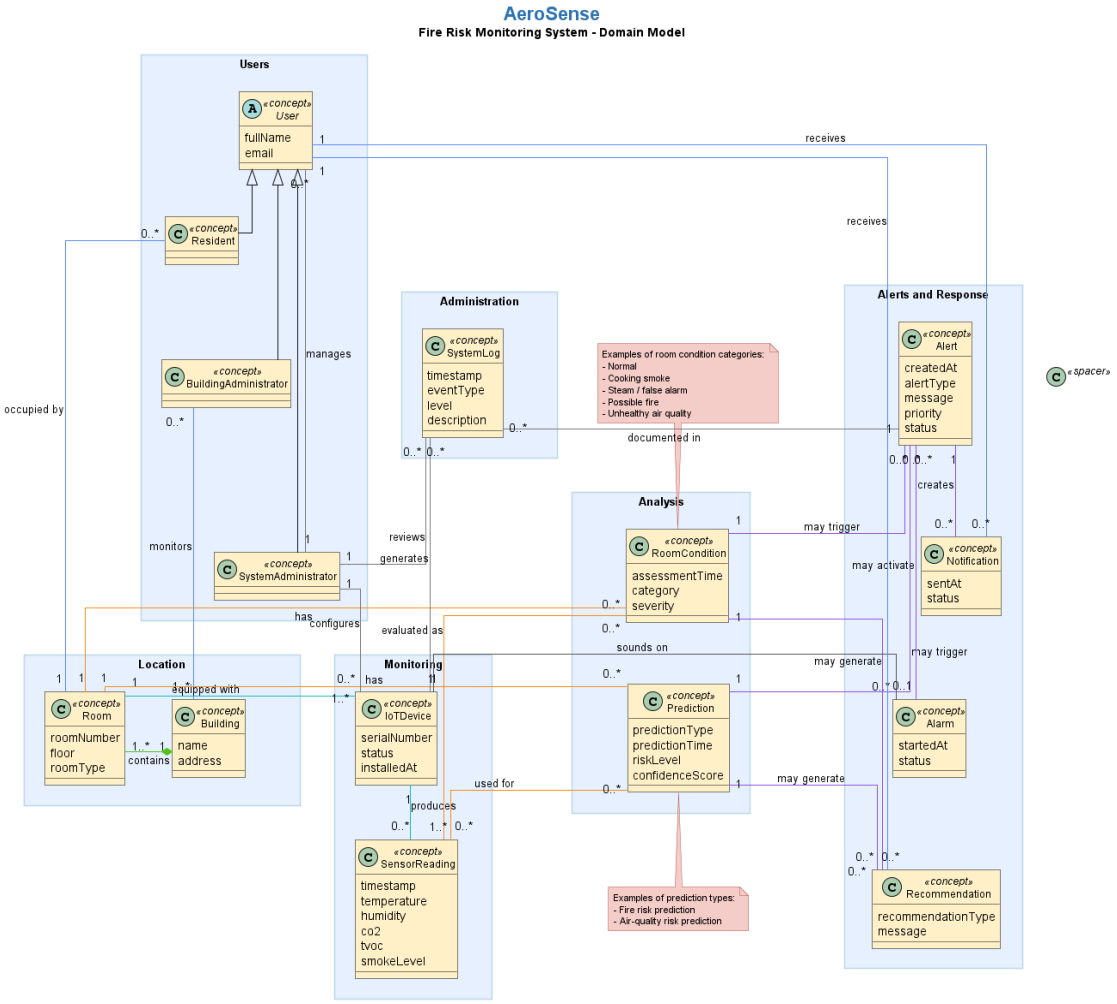

Figure 2 – AeroSense conceptual domain model

3.1.12 Analysis Summary

The analysis phase created the common foundation for the AeroSense system before the project was divided into subsystem-specific design and implementation work. During this phase, the main system actors were identified, user stories were created, and functional and non-functional requirements were defined.

The analysis also included a use case diagram to show the main interactions between the actors and the system, as well as a domain model to describe the most important concepts in the problem domain. These artifacts helped clarify what the system should support, how the different users interact with it, and which concepts are important for the system as a whole.

Overall, the analysis showed that AeroSense needs to support room monitoring, fire risk handling, air-quality advice, predictions, notifications, alarms, device management, and system administration. This shared understanding was then used as the basis for the following design section, where the overall architecture and the individual subsystem designs are described.

3.2 Design Overview including Cloud / Overall Architecture

In this section the report will look into how the system as a whole is structured before branching into the individual subteam sections. The SEP4 system is composed of four components: an IoT device layer, a central backend server, a machine learning inference server, and a web based frontend. Each component was designed with a clearly defined responsibility, and the interfaces between them were agreed upon early so that all four teams could develop in

parallel without blocking one another. The following subsections describe the overall architecture, the technologies used by each subsystem, how data moves through the system, and the main design decisions that shaped it.

3.2.1 Overall Architecture Diagram

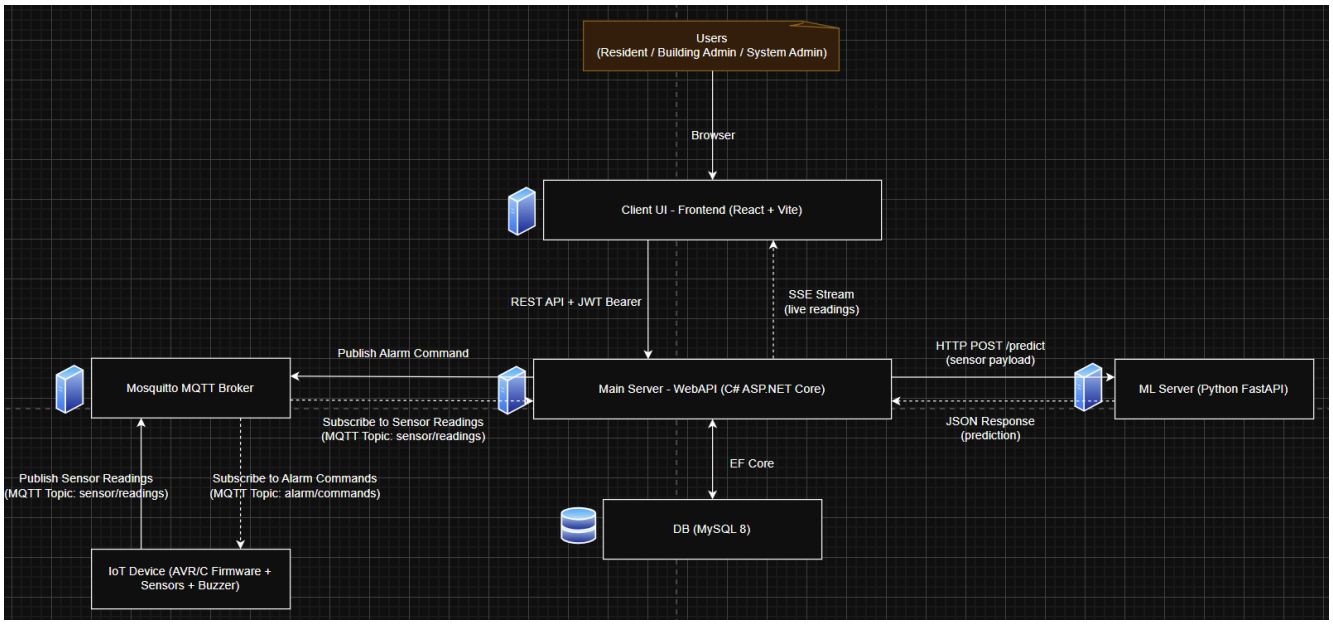

Figure 3 – Overall system architecture of the SEP4 fire detection system

3.2.2 Technology Stack

Each subsystem was developed using a technology stack suited to its role and to the expertise of the team responsible for it. The table below gives a summary of the main technologies per component.

The IoT layer runs firmware written in Embedded C on an ATmega2560 microcontroller (Arduino). The main backend server is built with C# using the ASP.NET Core framework. It uses Entity Framework Core as the object relational mapping layer on top of a MySQL 8 database, meaning all database interactions go through strongly typed model classes rather than raw SQL. The ML server is implemented in Python using the FastAPI framework, and relies on scikit-learn and pandas for model inference and data preparation. The frontend is a single page application built with React and Vite. On the infrastructure side, all server side services run as Docker containers hosted on Google Cloud Platform, and a Mosquitto broker handles all MQTT communication between the IoT device and the main server.

This structure ensures a clear separation of concerns, with each component responsible for a specific aspect of the system and no team needing to touch another team's stack to do their work.

3.2.3 Data Flow Between Subsystems

Sensor data originates at the IoT device, which reads values for temperature, humidity, CO2, TVOC, eCO2, and AQI at a defined interval and publishes them as a JSON payload to the Mosquitto broker. An MQTT-to-HTTP bridge subscribes to the iot/readings topic, receives each message, and forwards it via HTTP POST to the main server, which then persists it to the database.

After storing the reading, the main server forwards the sensor values to the ML server via an HTTP POST request to the `/predict` endpoint. The ML server processes the input and returns a classification result containing a

`predictedCategory`, a `confidenceScore`, and a `riskLevel`, which the main server then stores alongside the original reading and uses to evaluate whether an alarm should be triggered.

At the same time, the main server pushes the reading and its prediction result to any connected frontend clients through a Server Sent Events stream. This allows the dashboard to update in real time without the client having to repeatedly poll for new data. For historical data and configuration, the frontend uses standard REST API calls with a JWT Bearer token included in the request header.

The IoT device publishes sensor readings at a fixed interval of five seconds. This is implemented as a blocking delay at the end of the main loop and applies consistently regardless of sensor conditions.

3.2.4 Interface Contract Summary

The interfacing contract between all four teams was defined early in the project and kept stable throughout development. This approach allowed each team to build and test their component against a fixed specification without waiting for the others to finish.

The IoT device publishes sensor readings to the MQTT broker under a device specific topic. Each message contains a `sensorId`, a `timestamp`, and a `sensors` object with the six measured values. The device also subscribes to a separate topic (iot/alarm/{sensorId}) to receive alarm commands issued by the main server. The main server publishes CRITICAL when the ML result is Fire, WARN for elevated risk, and OFF otherwise.

The ML server exposes a prediction endpoint at `POST /predict`. The request body contains the full sensor payload and the response returns three fields: `predictedCategory` (one of `Normal`, `Cooking`, or `Fire`), `confidenceScore` (a float between 0 and 1), and `riskLevel`. A `/health` endpoint is also available to verify that the service is running.

The main server exposes a REST API for the frontend over HTTP. All requests require a valid JWT Bearer token issued on login. In addition to the standard request and response API, the server provides a persistent SSE endpoint that streams live sensor readings and prediction results to authenticated clients as they arrive. This contract remained stable from the initial integration onwards, meaning changes on one side did not force changes on the other.

3.2.5 Main Design Decisions

Several decisions made during the design phase had a significant effect on how the system as a whole came together.

In this project, MQTT was chosen for communication between the IoT device and the main server. MQTT operates over a lightweight publish and subscribe model, meaning the device does not need to know the address of the server and does not need to maintain a persistent connection to it. This proved useful given the ATmega2560's limited resources and the expectation that the device would be deployed in a setting where connectivity could be intermittent. The Mosquitto broker acts as the intermediary, decoupling the device from the backend entirely. (Banks and Gupta)

It was decided to containerise all server side services using Docker and deploy them on Google Cloud Platform. Each component runs in an isolated container, and the `docker-compose` configuration defines the startup order and internal networking. This approach allows the entire backend stack to be brought up or taken down with a single command and makes it straightforward to update any one component without touching the others. (Docker)

MySQL 8 was selected as the database for the main server. Its compatibility with Entity Framework Core allows for smooth integration with the application's data access layer, which enables efficient storage and retrieval of sensor

readings, device configurations, and alarm events. The choice of EF Core as the ORM layer additionally reduced the amount of boilerplate required and simplified schema migrations during development.

The ML inference logic was implemented as a standalone Python service rather than being embedded inside the main C# server. The reasons for this decision are covered in more detail in section 3.2.6.

3.2.6 Design Overview Summary

The sections that follow this one contain the detailed design documentation produced by each subteam. The following paragraphs provide a short transition into those sections by summarising the key design approach taken by each team.

IoT:

The IoT subsystem was built around a strict layered firmware architecture running on the ATmega2560 microcontroller. The application logic in main.c sits above a module and service layer that includes a sensors facade, WiFi driver, and MQTT client, all of which sit above low-level hardware drivers. This separation meant that sensor logic and communication code could be developed and tested independently on a native build target without requiring physical hardware. Readings from temperature, humidity, CO_2, TVOC, and electrochemical sensors are collected every five seconds, serialised as a JSON payload, and published to the MQTT broker. Alarm commands arriving from the backend are handled by a volatile flag in the main loop so that buzzer responses are never missed between sensor cycles.

Frontend:

The frontend was developed as a React web application built with Vite and structured around a central dashboard layout. The App.jsx component manages session state and decides whether the login page or the role-based dashboard should be shown. After authentication, the dashboard uses a persistent sidebar for navigation across monitoring and management pages. All data exchange with the backend uses a dedicated API service layer: REST endpoints handle data retrieval and a Server-Sent Events stream delivers live sensor updates to the interface without polling. Role-based access control is enforced in the interface so that residents, building administrators, and system administrators each see only the views and controls relevant to their role. The application is served through nginx inside a Docker container and deployed as part of the shared Docker Compose stack.

ML:

When designing the overall system an excellent use case for a separate inference service was encountered. The main server is written in C# and handles all the core business logic, but machine learning inference requires a different set of tools. Libraries such as scikit-learn and pandas, which were central to the model training and inference work, have no equivalent in the .NET ecosystem. It was decided to encapsulate all of the ML logic within a standalone Python FastAPI service, keeping it entirely separate from the main backend. This allowed the ML team to work with standard Python tooling throughout the full cycle from data preparation to deployment, without needing to adapt to a language that is less suited to that kind of work.

The interface between the two servers was kept intentionally simple: the main server sends a POST request with the sensor payload and receives a structured JSON response with the classification result. This contract proved useful throughout the project because it meant that model improvements and retraining could happen independently without requiring any changes on the backend side. The observer pattern seen in the previous semester's project showed the value of loose coupling between layers, and the same principle applied here at the service level rather than the class level.

It was acknowledged during design that running a separate service adds some operational overhead, both in terms of the additional container and the need to manage communication between services. However, given the clear division of expertise across teams and the requirements of the ML pipeline, the tradeoff was considered worthwhile. The FastAPI framework's automatic API documentation also proved useful during integration, as it gave other teams a way to inspect and test the prediction endpoint without reading through the source code directly.

3.3 IoT

3.3.1 IoT Design

Author: Matteo Maria De Filippis

3.3.1.1 IoT Responsibilities

The IoT subsystem is responsible for:

Responsibility	Implementation
Initializing and reading sensors	`sensors.c` facade over DHT11, MH-Z19B, ENS160
Building the outbound JSON payload	`build_payload()` in `sensors.c`
Connecting to Wi-Fi and the MQTT broker	ESP8266 AT commands + raw MQTT over TCP in `main.c`
Publishing readings	Topic `iot/readings`, QoS 0, approximately every 5 seconds
Subscribing to server alarms	Topic `iot/alarm/{sensorId}`
Driving the buzzer from alarm payloads	CRITICAL / WARN / OFF handling in `main.c`
Serial debug output	`uart_stdio` at 115200 baud

The IoT subsystem is not responsible for:

ML classification — the firmware sends `"classification":"Normal"` as a placeholder. MainServer calls the ML service and maps Fire → CRITICAL, while other categories are mapped to OFF through `AlarmMqttPublisher`.

Timestamps — timestamps are not set on the device. MainServer assigns the time when storing readings through `ReadingsController`.

Database, REST/gRPC communication with the frontend, and user authentication — these belong to the backend and frontend domains.

Choosing alarm logic from raw sensor thresholds on the device — the buzzer follows server MQTT commands, not local `alarm_logic`. That module is used only for Unity tests.

Dedicated smoke detector hardware — there is no smoke-specific driver in the firmware. Air-quality proxies come from the ENS160 sensor, including tVOC, eCO2, and AQI. The backend schema has an optional `SmokeLevel` field, but the current payload does not populate it.

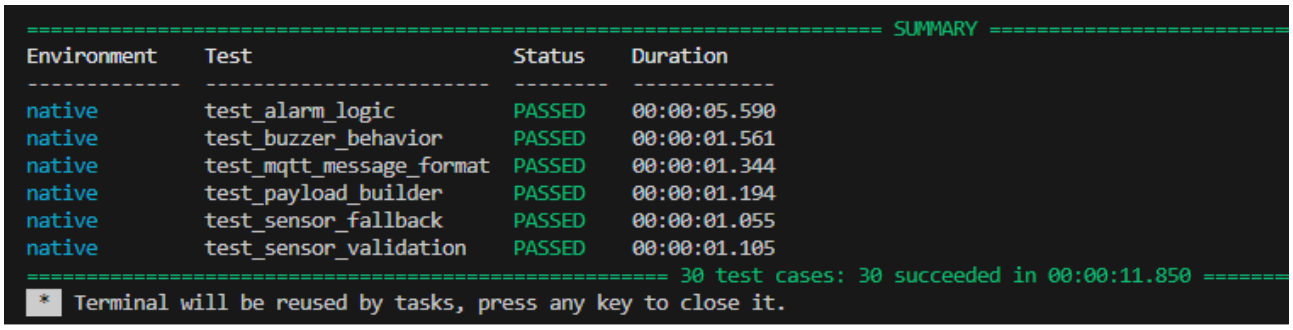

Figure 8 – Local IoT Unity test result showing 30 passed test cases

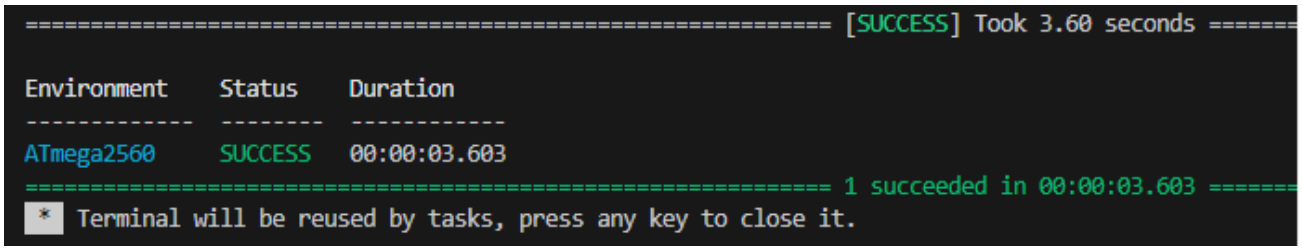

Figure 9 – ATmega2560 firmware build result showing successful build

The **IoT CI in GitHub Actions** was also updated to run the firmware build and Unity tests, and the GitHub Actions checks passed on the pull request. This gave extra confidence that the IoT code could be **built and tested automatically**, not only locally.

3.3.4.5 Test Limitations

The main limitation is that the Unity tests only verify pure logic modules. They do not prove that the full physical IoT system works on the real board. Real sensor readings, MQTT communication, backend integration, and buzzer behavior still need **manual hardware and integration testing.**

Another limitation is that several tested utility modules are not fully integrated into the current runtime firmware flow. For example, `main.c` still calls `build_payload()` from `sensors.c/h`, not `payload_builder_build()` from `payload_builder.c`. Because of this, the Unity tests show that the logic works in isolation, but not that all tested modules are used directly in production firmware

.

3.4 Backend

Author(s): Matteo Saccucci, Matteo Maria De Filippis, Hamsa Sheikhdon

3.4.1 Backend Design

The backend is implemented as MainServer, an ASP.NET Core REST API (System/MainServer). It sits between the IoT path (MQTT → HTTP bridge), the ML service (FastAPI), MySQL persistence, and the React frontend. It stores sensor readings and predictions, enforces JWT role-based access, streams live updates to the dashboard, and publishes MQTT alarm commands back to devices when ML classifies Fire.

3.4.1.1 Backend Responsibilities

MainServer responsibilities:

- Ingest sensor data

Accept JSON readings, store them, auto-register new sensorId devices.

- - ML inference

Send readings to MlServer (POST /predict) and store prediction results (category, confidence, risk).

- - IoT alarm

Publish MQTT to iot/alarm/{sensorId} (CRITICAL for Fire, OFF otherwise).

- - Frontend API

REST endpoints + SSE (/api/readings/stream) for real-time data.

- - Auth

BCrypt passwords + JWT roles (resident, building-administrator, admin).

- Ops

/health, logs at /api/logs, EF migrations on startup, env-based config.

Out of scope:

- Full incident workflow tables (EER).

- gRPC (uses REST/JSON instead).

3.4.1.2 Backend Architecture

Technology stack:

- - Runtime: .NET (ASP.NET Core)
- - ORM: EF Core + MySQL 8 (Pomelo)
- - Auth: JWT Bearer
- - Integration: HttpClient (ML), MQTTnet (Mosquitto)

- Deploy: Docker (main-server + MySQL + ML + MQTT + bridge + frontend)

Logical layers:

- - Controllers: API endpoints
- - Services: ML client, JWT, MQTT, streaming, logging
- - Data: DbContext, migrations
- - Models/DTOs: entities + request/response

Runtime flow:

- - IoT ⟶ MQTT (iot/readings)
- - Bridge ⟶ POST /api/readings
- - MainServer ⟶ save ⟶ ML ⟶ save ⟶ MQTT alarm ⟶ SSE
- - Frontend ⟶ /api via nginx + JWT auth

Architecture diagram:

- - Use/redraw AD_01.puml and include MySQL, Mosquitto, mqtt-http-bridge

3.4.1.3 API Design

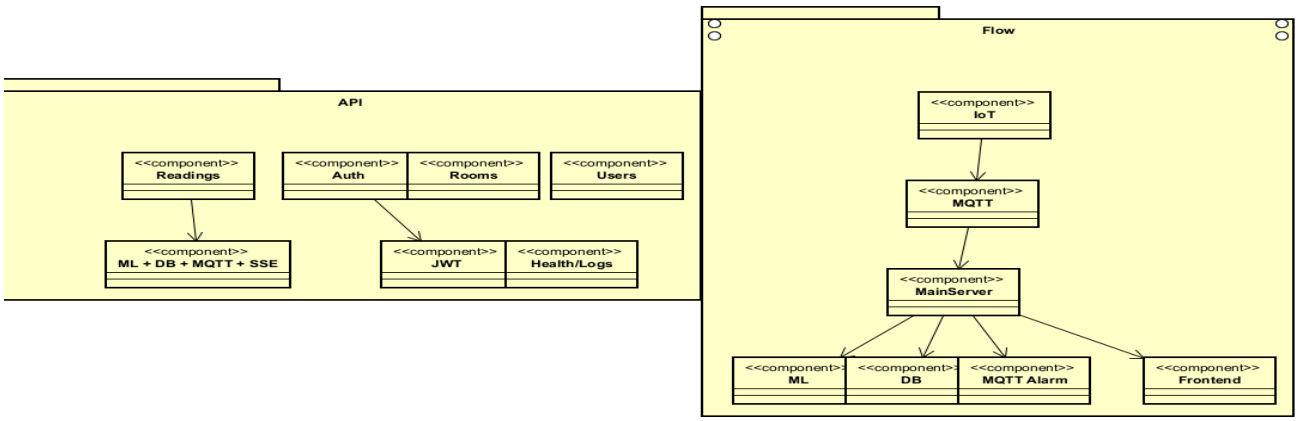

Figure 10 – API design overview.

3.4.1.4 Database Design

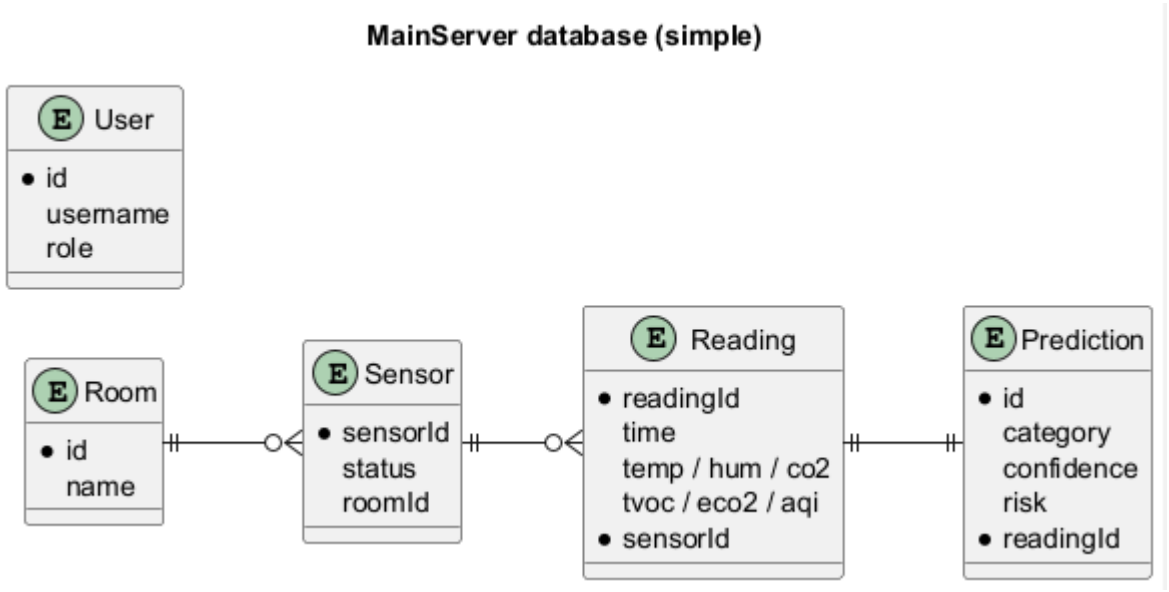

Figure 11 – Database schema design.

3.4.1.5 Backend Design Diagrams

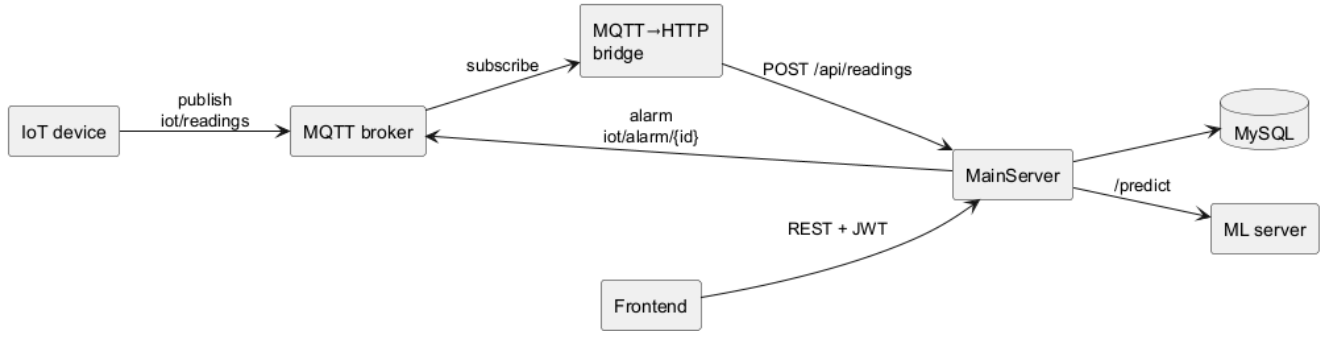

Figure 12 – Backend architecture overview.

3.4.1.6 Backend Design Decisions and Limitations

Decisions:

- - REST + JSON (frontend), SSE for live readings (no gRPC)
- - JWT roles: resident, building-administrator, admin
- - ML as separate service; predictions stored 1:1 with readings
- - MQTT alarms: iot/alarm/{sensorId} (Fire → CRITICAL, else OFF)
- - MySQL + EF Core; sensors auto-created on first reading
- - Docker/VPS stack: MQTT, bridge, MainServer, ML, frontend

Limitations:

- - MQTT bridge may lack JWT while ingest requires admin auth
- - Simplified DB (no Alert/Incident tables; alerts = filtered readings)
- - ML downtime → ingest fails (502) or missing prediction/alarm
- - Device “online” = no reading for ~120s (not heartbeat-based)
- - Resident access tied to sensorId/JWT claims (must align with frontend)

3.4.2 Backend Implementation

3.4.2.1 Implementation Overview

The final project structure of the backend is a single ASP.NET Core 10 project under `System/MainServer/`. It compiles to one executable that starts the web server, applies EF Core migrations to the MySQL database, and then begins accepting requests. There is also a separate `System/MainServer.Tests/` project that holds the unit tests. (Microsoft)

The main modules are the four controllers that make up the API layer, the four services that handle cross-cutting concerns, the EF Core data layer with its models and `AppDbContext`, and the DTO classes used to shape incoming and outgoing JSON. All configuration is loaded from environment variables at startup through `Program.cs`, with `DotNetEnv` used to read a `System/.env` file in development so that sensitive values like the database password and JWT key are never committed to the repository.

3.4.2.2 Domain and Service Layer

The domain models are simple POCOs with no business logic in them. `SensorReading` holds the raw sensor fields, `Prediction` holds the ML output linked one-to-one to a reading, `SensorDevice` represents a physical IoT device, `Room` groups sensors by location, and `User` holds authentication data and the optional `AssignedSensorId` that ties a resident to their sensor.

Business logic lives in the service layer. `JwtService` is responsible for generating tokens. It reads the JWT key from `JWT_KEY` environment variable first and falls back to `appsettings.json` only if the environment variable is not set. When generating a token for a resident with an assigned sensor it adds a `"SensorId"` claim alongside the standard role and identity claims. This claim is later read by `ReadingsController` to scope the resident's data access without any query parameter from the frontend being needed.

`MlClient` is a typed `HttpClient` that sends the sensor payload to `POST /predict` on the ML server and deserializes the JSON response into a `PredictionDto`. It retries the request once with a 150 ms delay before giving up, which handles transient connection issues during the brief period after the ML server container starts. If both attempts fail the controller returns a 502 rather than silently storing a reading with no prediction.

`ReadingsStreamHub` holds a `ConcurrentDictionary` of active SSE subscriptions. Each subscription is an unbounded .NET channel. When a new reading is published to the hub, it iterates the dictionary and writes the event to every channel whose `SensorId` filter matches. This structure ensures a clear separation of concerns, with the hub responsible only for fan-out and the controller responsible only for opening and closing the SSE connection.

`AlarmMqttPublisher` wraps an MQTTnet client and manages the connection lifecycle. It connects lazily on first publish and reconnects if the connection was lost. The topic used is `iot/alarm/{sensorId}` and the payload is one of three ASCII strings: `CRITICAL` when the ML model returns `Fire`, `OFF` for `Normal` or `Cooking`, and optionally `WARN` for medium risk if the `ALARM_PUBLISH_MEDIUM` environment variable is set. There is also an alarm test endpoint that allows a building admin to manually trigger a test publish without a real sensor reading.

3.4.2.3 Repository and Database Access

There is no separate repository layer. `AppDbContext` is injected directly into each controller that needs database access. This was a conscious decision to avoid unnecessary abstraction for a project of this size. The `DataModel` package in the previous semester's project used a similar direct approach and it resulted in a more easily modifiable structure, which carried over here.

EF Core handles all SQL generation. The relationships are configured in `OnModelCreating` rather than relying on conventions, which makes the foreign keys and cascade behaviors explicit. The one-to-one between `SensorReading` and `Prediction` is configured with `HasOne`/`WithOne` and `HasForeignKey<Prediction>`, and the `SensorDevice` to `Room` relationship uses `OnDelete(DeleteBehavior.SetNull)` so that deleting a room does not cascade to the sensor records.

Migrations are applied automatically at startup inside `ApplyMigrationsWithRepairAsync`. If the migration fails (for example because the database schema was manually changed) it falls back to `EnsureCreated`, which creates the tables directly from the model. This fallback was added after a situation in early development where a migration conflict caused the server to fail to start.

The database is MySQL 8 running in its own Docker container on the VPS. The connection string is assembled from five environment variables (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`) so that local development and the VPS deployment can point at different databases without touching any code. When running inside a container the connection string also appends `SslMode=None;AllowPublicKeyRetrieval=true` to handle MySQL's authentication plugin requirements in containerized environments.

3.4.2.4 API Layer

The controllers contain minimal logic. `ReadingsController.Post` is the most complex because it orchestrates the full reading pipeline: store reading, call ML, store prediction, broadcast to SSE, publish alarm. Each of those steps is delegated to a service. The controller only makes the ordering decisions and handles error responses if a step fails.

DTOs are used for all request and response bodies. `SensorReadingDto` accepts both a nested `sensors` object and flat top-level fields for backward compatibility. `ResolveSensorPayload` is a private helper that normalizes whichever format arrives into a `SensorPayloadDto` before it is mapped to the `SensorReading` model. This allowed the contract to evolve during the project without breaking existing IoT device firmware in the field.

Response objects are projected inline using anonymous types in LINQ `Select` clauses rather than dedicated response DTOs. This keeps the codebase smaller at the cost of slightly less explicit response shapes, which was considered an acceptable tradeoff given that the API is consumed by a single known frontend.

`AuthController` handles login, registration, and the `GET /api/auth/me` endpoint which the frontend calls after loading to rehydrate the session. New accounts are always created with the `resident` role. BCrypt is used for password hashing with the default work factor provided by `BCrypt.Net`.

`UsersController` is restricted to system admins entirely. It provides endpoints to list users, change roles, and assign or clear a sensor for a resident. The role change endpoint has guards to prevent an admin from demoting themselves if they are the only admin in the system, which avoids the situation of having a system with no one able to manage it.

3.4.2.5 Integration with ML, IoT, and Frontend

The Main Server integrates with three external subsystems, and each integration works differently.

For the ML Server, the integration is a simple synchronous HTTP call made by `MlClient` on every reading. The payload follows the interfacing contract described in section 3.2.4. The call is made inline during the `POST /api/readings` request and blocks until the ML server responds or times out after 5 seconds. If the ML server is unreachable the endpoint returns a 502 rather than storing the reading without a prediction, which keeps the data consistent.

For the frontend, the Main Server exposes two channels. The standard REST API handles all user interactions: fetching readings history, managing rooms and sensors, authentication. The SSE stream at `GET /api/readings/stream` handles live updates. The stream pushes a new event within milliseconds of a reading being processed, so the frontend dashboard updates in near real time without polling. CORS is configured through the `FRONTEND_ALLOWED_ORIGINS` environment variable so that both local development and the VPS deployment can be handled without changing code.

3.4.3 Backend CI/CD

Author(s): Matteo Saccucci, Matteo Maria De Filippis, Hamsa Sheikhdon

3.4.3.1 DevOps Considerations for Backend

Goals:

- Catch build/test failures before merge (dev, main, PRs)

- Package API as a Docker container with env-based config (DB, ML, MQTT)

- Deploy full stack on a single VPS (MySQL, ML, MQTT, bridge, MainServer, frontend)

3.4.3.2 CI Pipeline

CI Jobs:

- MainServer (.NET, ubuntu-latest)

- - dotnet restore & build (MainServer.Tests)
- - dotnet test + Coverlet (≥5% coverage, exclude migrations)
- - Upload TRX + coverage artifacts

- Build Images

- - docker build System/MainServer (tag: kamtjatka/main-server:ci, no push)

- Parallel Jobs

- - MlServer: ruff + pytest
- - IoT: PlatformIO build/test
- - Frontend: Docker build

- Deploy

- - Runs after: mainserver, mlserver, docker

3.4.3.3 CD / Deployment

- Trigger: push to dev or main (after CI passes)

- SSH via GitHub secrets

- git fetch + checkout + reset --hard origin/<branch>

- docker compose (docker-compose.vps.yml)

- - stop/remove containers
- - up -d --build
- - prune images

VPS Stack:

- - MySQL

- - ml-server
- - main-server
- - mqtt-broker
- - mqtt-http-bridge
- - frontend

Local Dev:

- - Use System/docker-compose.yml (main-server only)
- - Configure DB_HOST / ML_SERVER_URL to point to VPS

3.4.3.4 Secrets and Configuration

Secrets & Config:

- GitHub Actions secrets:

- - VPS_SSH_KEY
- - VPS_HOST
- - VPS_USER
- (not stored in repo)

- Runtime environment (.env / docker-compose on VPS):

- - DB_*
- - ML_SERVER_URL
- - JWT_KEY / Jwt:*
- - ALARM_MQTT_HOST
- - ALLOW_ALARM_TEST
- - FRONTEND_ALLOWED_ORIGINS
- - etc.

- MainServer loads .env via DotNetEnv (Program.cs)

- Security:

- - No passwords/keys in git
- - Developers use local .env templates

3.4.3.5 What Worked Well and Less Well

Results:

Worked well:

- - Single CI workflow for entire project
- - Automatic VPS deploy on dev/main after CI success
- - Docker build verifies images compile
- - Tests + coverage gate for MainServer

Worked Less Well:

- - Deployment uses SSH + docker compose (no registry-based CD)
- - CI does not push/version images (no image registry pipeline)
- - Monolithic deploy: all services restart together
- - Low test coverage threshold (5%) for backend
- - MlServer/IoT failures can block backend-only deployments

3.4.4 Backend Test

Author: Waqar Ahmed Khan

3.4.4.1 Test Strategy

Backend testing focused on the prediction path exposed by MainServer through POST /api/readings. In this path, the backend accepts a sensor reading, stores it, calls MLServer through MlClient, stores the prediction returned by the service, and sends the prediction back to the client. This flow is important because a failure at the HTTP boundary or persistence layer would prevent reliable fire-risk information from reaching the rest of the system.

The test strategy combined automated tests and manual black-box verification. Automated tests isolated the backend behavior by using fake ML HTTP responses and an in-memory database, so they could run safely without Docker or the shared MySQL database. Manual API testing then exercised the deployed interface of the complete local Docker stack to verify the real MainServer-to-MLServer-to-MySQL flow.

3.4.4.2 Unit Tests

The MlClient service was unit tested with xUnit because it is the MainServer boundary responsible for sending readings to MLServer. A custom fake HTTP message handler intercepted outgoing requests and returned controlled prediction responses. This allowed the test to verify the request method, the /predict route, the nested sensor payload and the mapping of predictedCategory, confidenceScore and riskLevel without starting the Python service.

Additional MlClient tests covered fault-handling behavior: a temporary ML failure triggers one retry; nullable TVOC, eCO2 and AQI values are converted to the expected fallback values; and repeated ML unavailability results in an HttpRequestException. The extract below illustrates how the outgoing contract was checked.

Listing 3.4.4-1. Selected xUnit assertions for the MainServer-to-MLServer request contract.

```
Assert.Equal(HttpMethod.Post, handler.Request!.Method);
Assert.Equal("/predict", handler.Request.RequestUri!.AbsolutePath);
var sensors = body.RootElement.GetProperty("sensors");
Assert.Equal(1000, sensors.GetProperty("tvoc").GetDouble());
```

3.4.4.3 Integration Tests

ReadingsControllerTests exercised the behavior of POST /api/readings across the controller, MlClient and persistence operations. The tests used a real MlClient with a fake HTTP response and an Entity Framework Core in-memory database. Therefore, they verified the storage behavior of the controller without modifying the real MySQL database or requiring running containers.

In the successful scenario, a Fire prediction was returned and the test confirmed that one sensor, one reading and one linked prediction were stored. In the failure scenario, MLServer was simulated as unavailable: MainServer returned HTTP 502, retained the received reading for traceability, and did not create a prediction record. The test setup also restores the ALARM_MQTT_HOST environment variable after construction, avoiding global test-state leakage.

Listing 3.4.4-2. Selected controller assertions for persistence and ML failure handling.

```
// Successful prediction persists both records
Assert.Equal(1, await db.Readings.CountAsync());
Assert.Equal(1, await db.Predictions.CountAsync());
// Failed ML response returns 502 and no prediction record
Assert.Equal(StatusCodes.Status502BadGateway, status.StatusCode);
Assert.Equal(0, await db.Predictions.CountAsync());
```

Table 3.4.4-1. Automated tests added for the MainServer prediction flow.

Test method	Verified behavior	Result
MlClient: payload/response	POST /predict request contract and DTO mapping	**Passed**

MlClient: retry	One retry following an initial ML HTTP failure	**Passed**
MlClient: null defaults	TVOC/eCO2/AQI fallback values 0, 0 and 1	**Passed**
MlClient: unavailable	Exception after repeated ML failure	**Passed**
Controller: success	Reading and linked prediction stored; response returned	**Passed**
Controller: ML failure	HTTP 502; reading stored; prediction not stored	**Passed**

3.4.4.4 Manual API Testing

Manual black-box testing was performed against the running Docker-based system by calling only public MainServer HTTP endpoints. Both MainServer and MLServer health endpoints returned status ok. Three requests were then sent to POST /api/readings, representing normal indoor conditions, cooking activity and a fire-like condition. MainServer returned the expected prediction class and risk level for all three requests.

After the API calls, persistence was checked directly in the local Docker MySQL database by joining the Readings and Predictions tables. The complete JSON request bodies and SQL verification query are included in Appendix D so that the black-box test can be reproduced.

Table 3.4.4-2. Results of the manual black-box prediction tests.

Sensor ID	Profile	Prediction	Risk level	Confidence	Result
9001	Normal	Normal	Low	0.84	Passed
9002	Cooking	Cooking	Medium	0.85	Passed
9003	Fire	Fire	High	0.63	Passed

3.4.4.5 Test Results and Limitations

The MainServer automated test suite completed successfully with **28 passed** tests, **0 failed** tests and **0 skipped** tests. The relevant new coverage includes four MlClient unit tests and two ReadingsController integration-style tests. The repository CI workflow is configured to run MainServer restore, build and test steps on pull requests to Dev or main and to collect line coverage.

The manual black-box checks also passed for all three sensor profiles and verified persistence in local Docker MySQL. The scope of this backend test contribution is limited to the MainServer-to-MLServer prediction path and database persistence. Frontend rendering, physical IoT alarm behavior, authentication, load testing and complete CI-driven end-to-end testing remain outside this test scope. Further negative-input tests for POST /api/readings would be useful once the intended validation rules are agreed by the backend team.3.5 Machine Learning

3.5.1 ML Data Exploration

Author: Waqar Ahmed Khan

3.5.1.1 Dataset Overview

AeroSense used two complementary data sources because indoor monitoring must distinguish normal conditions, cooking activity and dangerous smoke. The **IoT dataset** supplied Normal and Cooking examples, while the **Blattmann smoke-sensor dataset** supplied Normal and Fire observations. After preparation, the sources were merged into one modelling table with **60,212 readings**: **40,823 Fire (67.8%)**, **14,671 Normal (24.4%)** and **4,718 Cooking (7.8%)**. (Blattmann)

held-out test set evaluated in CI would provide stronger guarantees, but was not implemented within the project scope. The drift monitor in `ImprovedModels/RFModel/drift_monitor.py` partially addresses this concern for production use, but it is an offline tool and is not wired into the automated pipeline.

Another limitation is that there are no integration tests that exercise the full path from the Main Server calling the ML Server. The ML tests operate entirely within the FastAPI test client, and the backend tests mock the ML response. This means a mismatch between what `MlClient.cs` sends and what `main.py` expects would only be caught at runtime. The interfacing contract was cross-checked manually during development, but an automated contract test would be a clear improvement.

3.6 Frontend

3.6.1 Frontend Design

Author: Rebeca Proskovcova

3.6.1.1 UI Goals

The frontend part of AeroSense is responsible for presenting fire risk and air quality information to the user in a clear and understandable way. Since the system works with sensor readings and possible warning situations, the interface had to be designed so that important information can be found quickly. The dashboard gives users access to live readings, room status, recommendations, historical samples, payload data, alerts, and role-based management features.

3.6.1.2 Figma Dashboard Design

The dashboard design was created to provide a clear overview of sensor readings, room conditions, warnings, and user-specific functionality. The final design follows a dark dashboard style with a sidebar on the left, the current page title at the top, and content cards in the main area. This layout was chosen because it allows users to move between the main parts of the system while keeping the most important monitoring information visible.

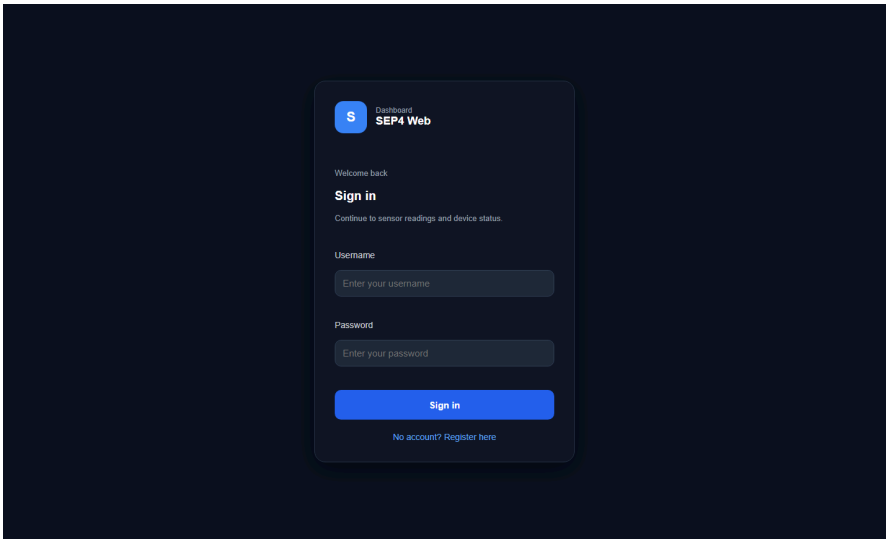

Figure 19 – Login page.

The login page is the first screen of the system. It contains the SEP4 Web branding, username and password fields, and the sign-in button. The page uses the same dark visual style as the rest of the application to keep the design consistent.

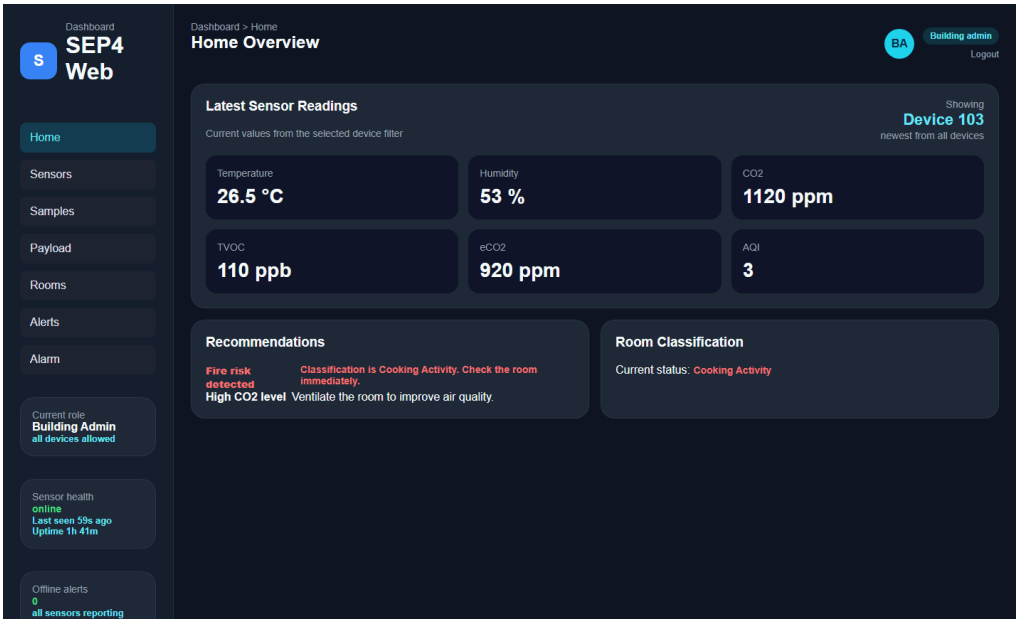

Figure 20 – Dashboard overview.

The dashboard overview shows the latest sensor readings from the selected device. It displays values such as temperature, humidity, CO2, TVOC, eCO2, and AQI. The page also shows recommendations and the current room classification, so the user can quickly understand whether the room condition is normal or requires attention.

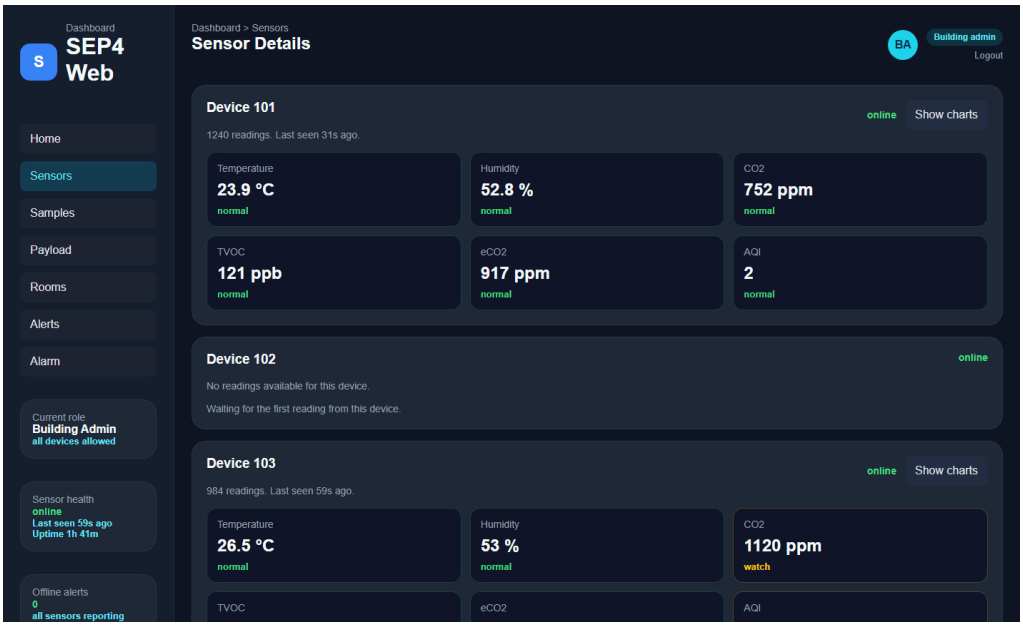

Figure 21 – Sensor details page.

The sensor details page shows readings for individual devices. It allows the user to inspect sensor values more closely and compare the status of different devices. The page also includes chart functionality for viewing trends when readings are available.

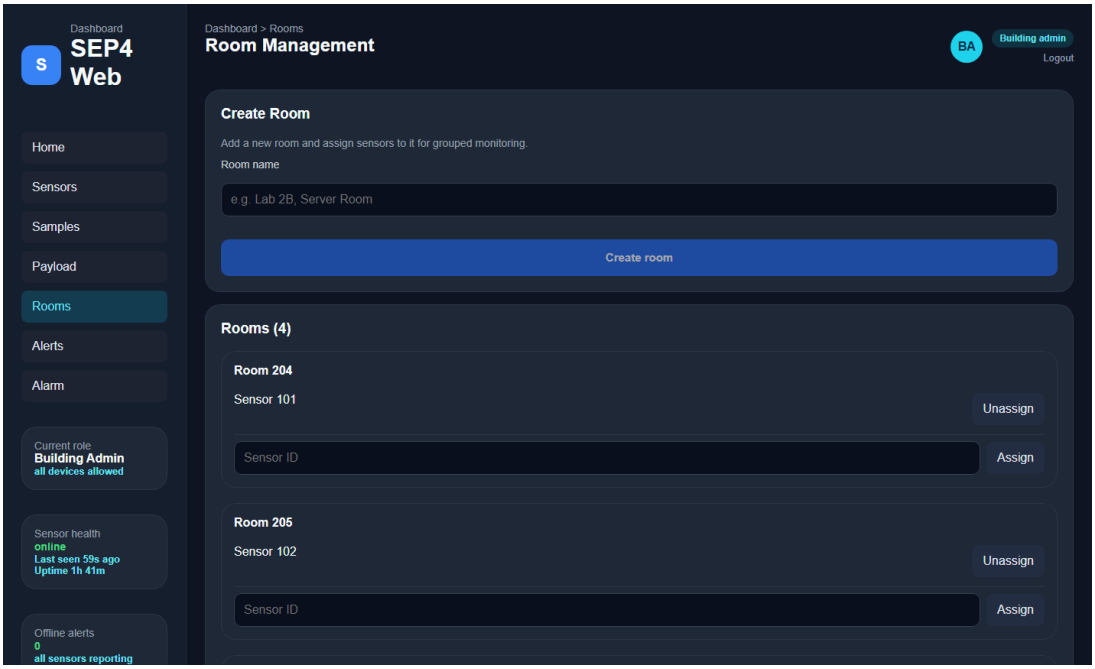

Figure 22 – Room management page.

The room management page is used to create rooms and assign sensors to them. This is mainly intended for administrators, because it supports the connection between physical rooms and sensor devices. The page shows existing rooms, assigned sensors, and controls for assigning or unassigning sensors.

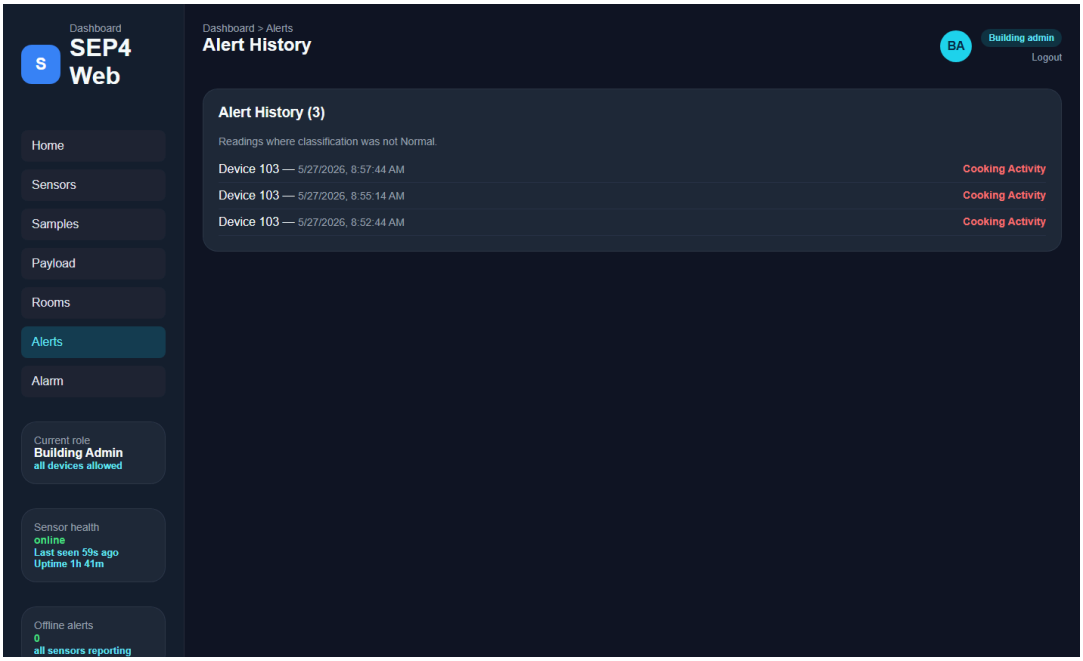

Figure 23 – Alert history page.

The alert history page shows readings where the classification was not normal. This helps administrators review warning situations, such as cooking activity or other abnormal sensor states. The warning labels make it easier to identify which device caused the alert and when it happened.

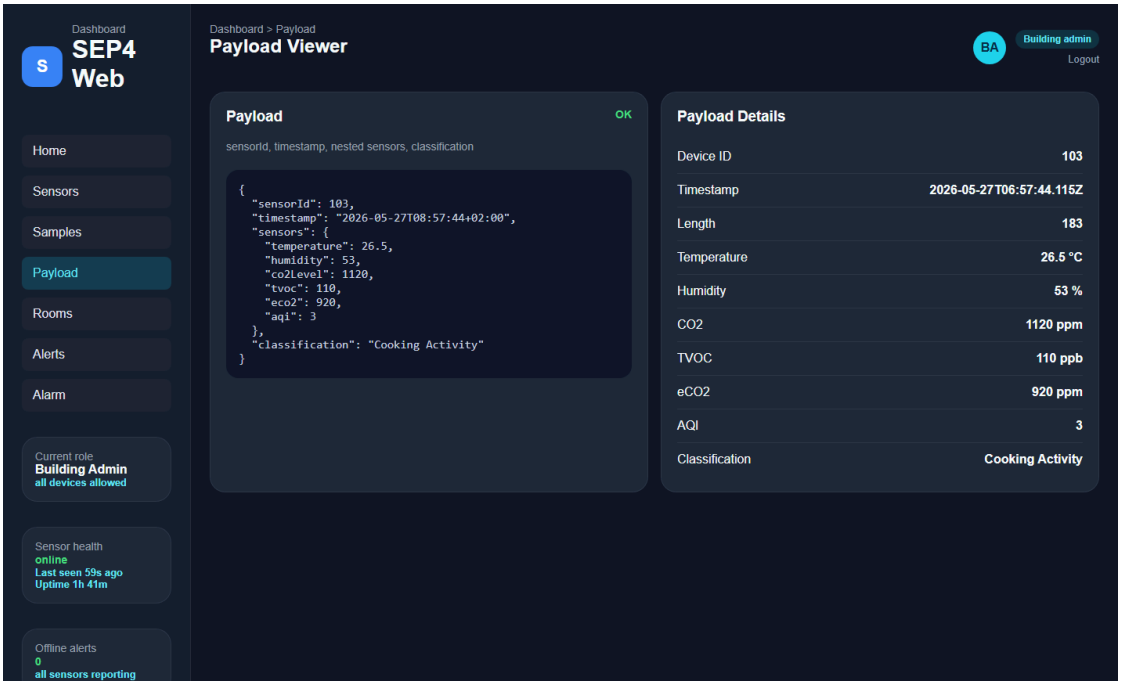

Figure 24 – Payload viewer.

The payload viewer displays the JSON payload sent by the sensor system. This view is useful for checking the exact data structure, including sensor ID, timestamp, sensor values, and classification. It helps show how the frontend represents the data received from the backend.

Overall, the dashboard design focuses on clear navigation, readable sensor data, role-based access, and simple monitoring of room conditions. The main components are the sidebar, topbar, sensor reading cards, recommendation panel, room classification section, room management view, alert list, and payload viewer.

3.6.1.3 Frontend Architecture

The frontend is implemented as a React application. The application is structured around reusable components, API service functions, and role-based views. The root application handles authentication state and decides whether the user should see the login page or the dashboard. After login, the user session is stored locally and used to determine which dashboard views are available.

The main dashboard consists of several pages or views, including Home, Sensors, Samples, Payload, Rooms, Alerts, Alarm, Users, Devices, and Logs. Not every user can access all views. The available navigation items depend on the user role. Residents have limited access to their assigned sensor or room information, while building administrators and system administrators can access more system-wide information.

The frontend uses components such as Sidebar, Topbar, OverviewCard, StatCard, SampleCard, PayloadCard, LineChart, RoomsView, and StatusScreen. These components help divide the interface into smaller parts and make the dashboard easier to maintain. Cards are used to display sensor values and summaries, while chart components are used to visualize historical sensor data.

Communication with the backend is handled through API service functions. These functions are responsible for login, registration, loading readings, loading devices, fetching alerts, managing rooms, managing users, sending alarm test commands, and loading system logs. The frontend also uses a readings stream to receive live updates from the backend, which allows the dashboard to update when new sensor data is received.

State management is handled mainly with React state and effects. This includes storing the active view, selected sensor, loaded readings, device status, user session, loading states, and error states. This approach was suitable for the prototype because the application state was not large enough to require a more complex state management library.

3.6.1.4 Frontend Design Diagrams

To explain the structure and behavior of the frontend, two diagrams were created: a component diagram and sequence diagram. The component diagram gives an overview of the main frontend parts and how they are connected. It shows that the application starts in App.jsx, which handles the user session and decides whether the login page or the dashboard should be displayed. After login, the dashboard becomes the main layout of the application and contains the sidebar, topbar, role-based views, reusable UI components, and the API service layer.

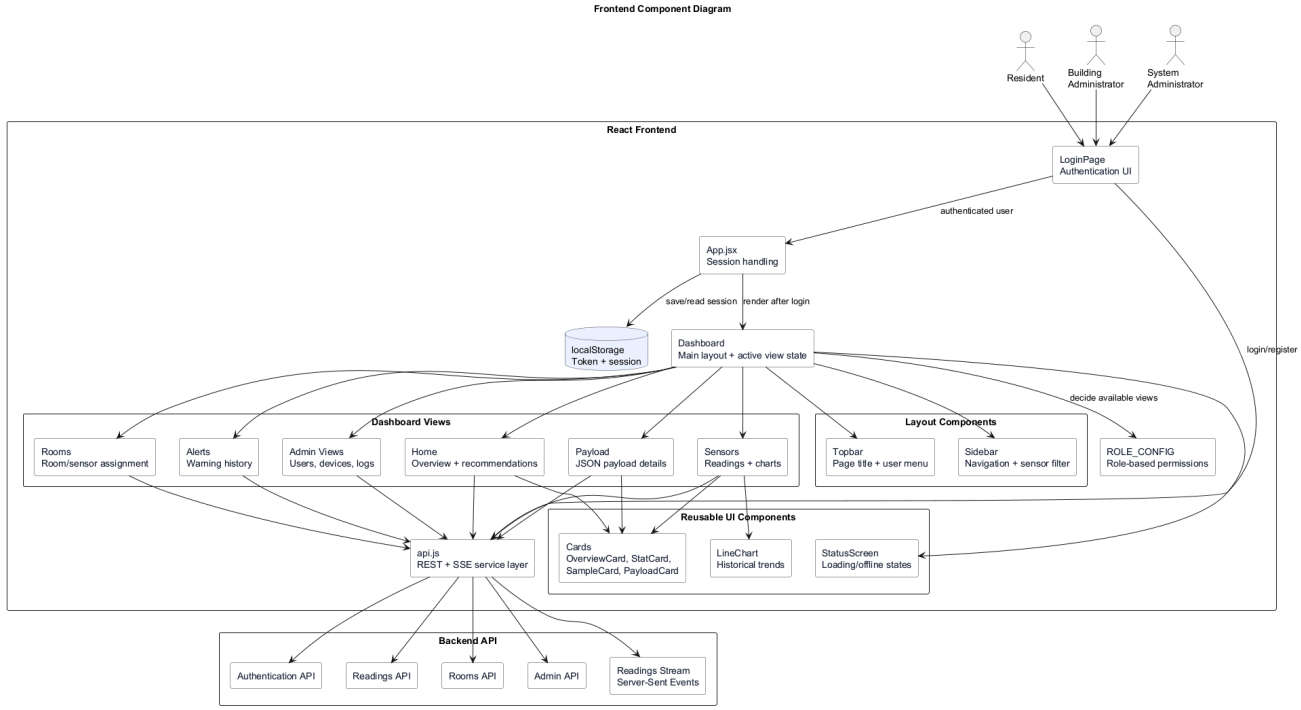

Figure 25 – Frontend component diagram showing the structure of the React application and its connection to the backend API.

The component diagram also shows how the frontend communicates with the backend through api.js. This service layer is responsible for authentication, sensor readings, room management, alerts, users, devices, logs, and live readings through Server-Sent Events. The diagram helps show that the frontend is divided into smaller components instead of being built as one large page.

The sequence diagram focuses on the login and sensor data flow. It shows how the user enters login information, how the frontend sends the request to the backend, and how the returned token and user role are stored in local storage. After authentication, the dashboard is rendered based on the user role. The dashboard then requests sensor readings and connects to the live readings stream so that new data can update the interface automatically.

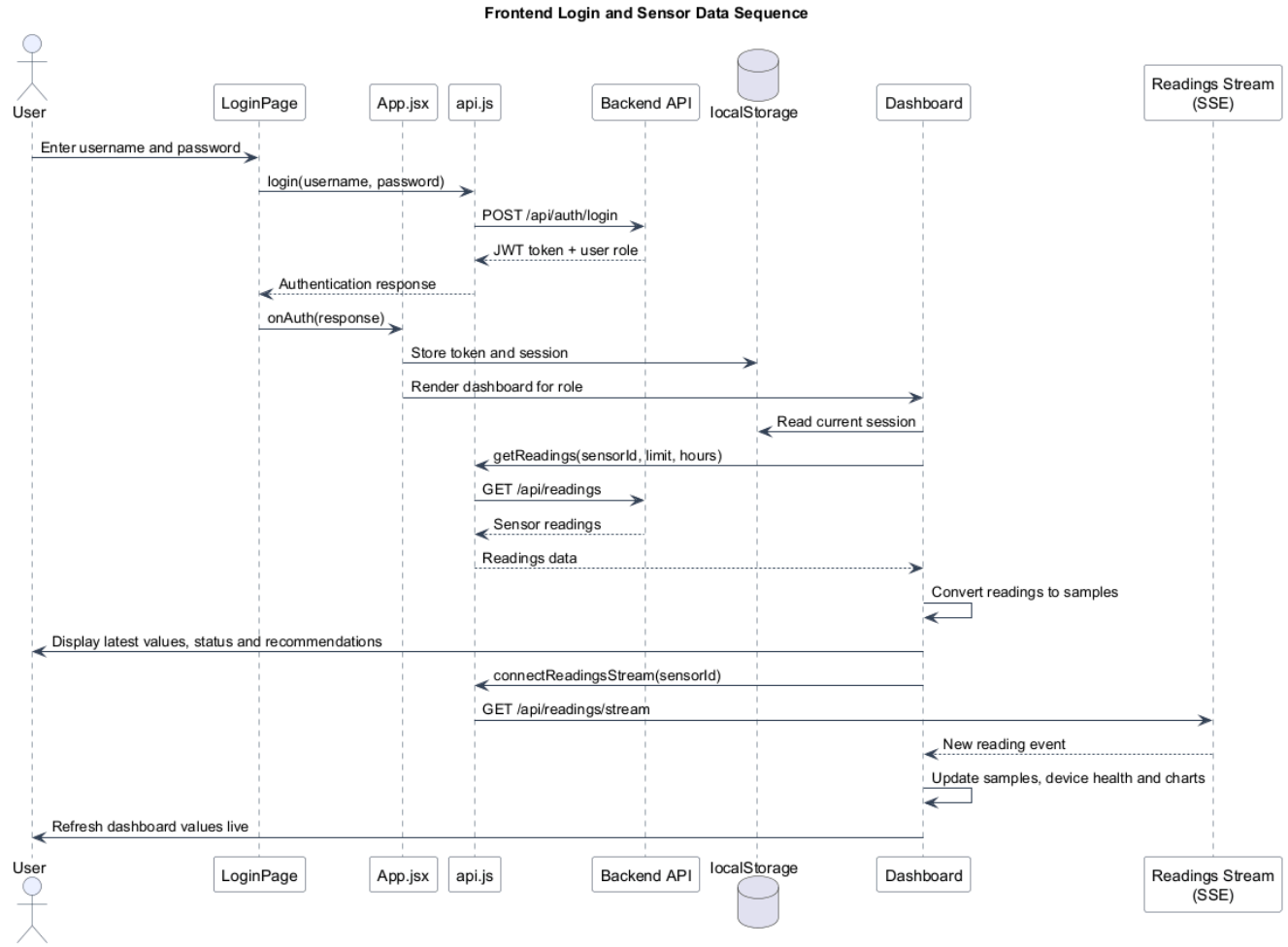

Figure 26 – Frontend login and sensor data sequence diagram showing authentication, dashboard loading, API communication, and live sensor updates.

These diagrams were useful during the design phase because they made the frontend structure easier to understand and helped clarify how the user interface communicates with the backend. They also show how the frontend supports role-based access and live monitoring, which are important parts of the AeroSense system.

3.6.1.5 Frontend Design Decisions and Limitations

One important design decision was to make the dashboard role-based. This means that the same frontend application can be used by residents, building administrators, and system administrators, while still showing different features depending on the user's permissions. This made the system more flexible and closer to a real building monitoring system.

Another design decision was to use a dashboard layout with a sidebar. This was chosen because AeroSense contains several monitoring and management pages, and a sidebar makes navigation clear when the user needs to move between views. Cards and charts were used because they make sensor values easier to scan compared to displaying all data in plain tables.

The frontend also includes simple recommendation logic based on sensor values and classification results. This was done to make the application more helpful for users, since raw values alone may not always explain what action should be taken.

There are also some limitations. The frontend depends on the backend being available, so if the API is offline, the dashboard cannot load live data. The system is also still a prototype, so the interface focuses on the most important monitoring and management features rather than a fully polished production system. Some advanced features, such as detailed notification settings, advanced filtering, or full building map visualization, are outside the current scope but could be added in future work.

3.6.2 Frontend Implementation

Author: Rebeca Proskovcova

3.6.2.1 Implementation Overview

The frontend implementation of AeroSense was developed as a React web application. Its main purpose is to give users a clear interface for monitoring room conditions, sensor readings, alerts, and system status. The implementation focuses on making the collected sensor data understandable for both residents and administrators. Instead of only showing raw values, the interface presents the data through dashboard cards, status labels, charts, payload details, and recommendations.

3.6.2.2 Dashboard and Room Conditions

The dashboard is the main view of the frontend. It shows the latest sensor readings, including temperature, humidity, CO2, TVOC, eCO2, and AQI. These values are displayed as cards so the user can quickly understand the current room condition.

The system also displays the current classification of the room, such as whether the room condition is normal or if there is a possible warning situation. Based on the latest readings, the frontend generates recommendations for the user. For example, if CO2 is high, the dashboard can recommend ventilating the room. If the classification is not normal, the interface highlights the situation so the user knows that the room should be checked.

Historical data is shown through sample lists and charts. This allows users to see how the sensor values changed over time, instead of only seeing the newest reading. The alerts view displays readings where the classification was not normal, helping administrators review previous warning situations.

The Rooms view allows administrators to see rooms and assigned sensors. This supports grouped monitoring, where sensor readings can be connected to specific rooms. The frontend also includes device status information, such as whether a sensor is online or offline and when it was last seen.

3.6.2.3 Authentication and Routing

Authentication is implemented through a login page where the user enters a username and password. The login request is sent to the backend, and if the response is successful, the frontend stores the token and user information locally. This session information is then used to keep the user logged in after refresh.

The frontend uses role-based access to decide which views and controls should be available. A resident has limited access and mainly sees information related to their assigned sensor or room. A building administrator can access broader monitoring features such as rooms, alerts, and alarm controls. A system administrator has access to additional management views such as users, devices, and logs.

The application does not use separate URL routes for each page. Instead, it uses an internal dashboard state to switch between views. The sidebar controls the active view, and the dashboard renders the selected page based on that state and the user's permissions.

3.6.2.4 API Integration

The frontend communicates with the backend through the API service layer in api.js. This file contains the functions used by the React components to send requests and receive data. The frontend uses API calls for login, registration, sensor readings, devices, alerts, rooms, users, alarm testing, and logs.

Sensor readings are loaded from the backend and converted into frontend sample objects before being displayed in the dashboard. The frontend can request readings for all sensors or for one selected sensor, depending on the user role and selected device filter.

The frontend also connects to a live readings stream using Server-Sent Events. This allows the dashboard to receive new sensor readings automatically without requiring the user to refresh the page. When a new reading arrives, the dashboard updates the sample list, device status, charts, and latest displayed values.

Authentication tokens are included in API requests where needed, so protected backend endpoints can identify the current user.

3.6.2.5 Error Handling and Loading States

The frontend includes loading and error states to make the application more stable and user-friendly. When the dashboard is loading data, a loading screen is shown instead of an empty or broken interface. If the backend API is unavailable, the frontend displays an offline status screen explaining that the dashboard cannot reach the backend.

Individual views also handle missing or unavailable data. For example, if no readings exist for a selected device, the Sensors view shows a message instead of failing. If alerts, users, rooms, or logs cannot be loaded, the interface displays an error message or fallback content.

This error handling is important because the system depends on several connected parts, including the backend, IoT devices, and live data stream. By showing clear loading and error states, the frontend remains understandable even when some data is delayed or unavailable.

3.6.3 Frontend CI/CD

Author: Soma Nagy, Piotr Gała

3.6.3.1 DevOps Considerations for Frontend

The main DevOps goal for the frontend was to ensure that every change pushed to the repository could be verified and deployed to the live system automatically, without requiring manual steps from the developer. Because the frontend is a user-facing component that depends on the backend API being available, the priority was on keeping the deployed version consistent with the latest stable code on the Dev and main branches. Automated frontend testing, production build validation, and Docker image building were used to make the process reproducible, and SSH-based deployment to the VPS was configured so that a successful push would result in the updated frontend being live without any manual intervention.

3.6.3.2 CI Pipeline

The CI pipeline for the frontend is defined in a shared GitHub Actions workflow file that covers all subsystems of the AeroSense project. The pipeline triggers automatically on every push or pull request targeting the Dev or main branches, and can also be started manually using the workflow_dispatch trigger. The frontend has its own CI job, which installs dependencies with npm ci, runs Vitest unit/component/API tests, builds the Vite production bundle, installs Playwright Chromium, and runs Playwright black-box browser tests. The shared Docker job also builds the frontend image.

3.6.3.3 Deployment

Deployment of the frontend is handled by the deploy job in the pipeline, which runs only when a push event occurs on the Dev or main branch, not on pull requests. The job depends on the mainserver, mlserver, frontend, and docker jobs all completing successfully before it is allowed to run, ensuring that the full system is tested and buildable before any deployment takes place.

3.6.3.4 Tools Used and Not Used

GitHub Actions was used as the CI/CD platform because the project repository was already hosted on GitHub, making it straightforward to define and trigger workflows without any additional infrastructure. npm was used for frontend dependency installation and scripts, Vitest for unit/component/API tests, and Playwright for browser-based black-box tests. Docker and Docker Compose were used for both building and deploying the frontend, which made the deployment environment consistent and independent of the specific configuration of the VPS. The docker/setup-buildx-action and docker/build-push-action were used to handle image building efficiently within the pipeline.

3.6.3.5 What Worked Well and Less Well

The automated deployment pipeline worked well in practice. Once a change was pushed to the Dev or main branch and all pipeline jobs passed, the updated frontend was available on the VPS without any manual steps. This removed the risk of forgetting to deploy or deploying an incorrect version, and meant that the live system always reflected the latest merged code. The dependency chain in the pipeline, where the deploy job only runs after the docker, mainserver, mlserver, and frontend jobs all succeed, also provided a basic safety mechanism that prevented a broken build or failing frontend test suite from reaching the

server. The main limitation is that the pipeline does not currently run a post-deployment smoke test against the live VPS URL, so final live verification still has to be done manually.

3.6.4 Frontend Test

Author: Piotr Gała

3.6.4.1 Test Strategy

Front-end testing focused on the behavior that creates the highest risk for users: authentication, role-based access, API communication, dashboard rendering, alarm controls, error handling, and responsive layout. The goal was not to test React itself, but to check that the application presents the correct views and calls the backend with the expected requests.

The test strategy used three levels. Unit tests covered pure access-control logic. Component tests covered isolated UI parts such as the login page, role selector, user menu, restricted notice, and alarm controls. API service tests mocked fetch and verified request URLs, HTTP methods, authorization headers, request bodies, and error messages. Playwright black-box tests then checked the application from the user's point of view in a real browser.

3.6.4.2 Unit and Component Tests

Unit tests were written with Vitest for the access-control module. These tests check that unknown roles fall back safely to the resident role, that role aliases are mapped correctly, that resident and admin permissions are different, and that the default dashboard view is selected from the allowed views.

Component tests were written with React Testing Library. The LoginPage test verifies that the sign-in form renders with username, password, and submit controls. LoginRoleSelect verifies that resident, building administrator, and system admin roles are visible and selectable. UserMenu verifies that the current role is shown and that logout calls the provided callback. RestrictedNotice verifies both default and custom access-denied messages.

The AdminControls component tests cover the alarm test UI. They verify that alarm buttons are disabled when all devices are selected, disabled while a request is busy, and that clicking Critical pattern, Short warn, and Silence calls the handler with critical, warn, and off respectively. This is important because these buttons can publish alarm commands to the IoT buzzer path through the backend.

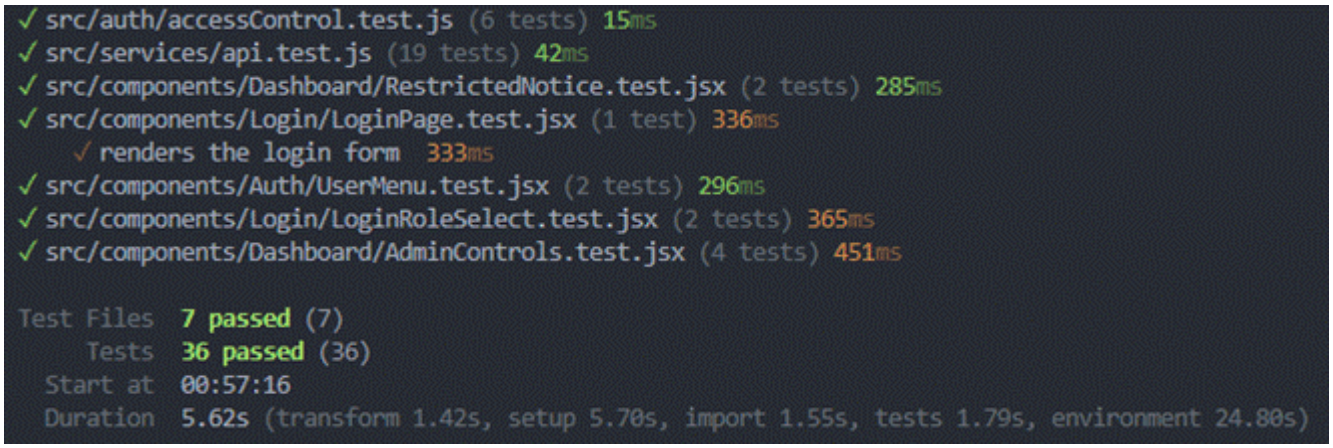

Figure 27 – Vitest results for frontend unit, component, and API service tests.

3.6.4.3 Integration Tests

The frontend API service tests check the integration boundary between React and the backend contract. The tests mock fetch, then verify that getReadings builds the correct query string, includes the Authorization header when a token exists, and throws a readable error when the backend fails. Similar tests cover devices, alerts, rooms, users,

logs, authentication, registration, role changes, sensor assignment, room sensor assignment, alarm tests, and the readings stream URL.

These tests are useful because most frontend bugs in this project would happen at the API boundary: a wrong endpoint, missing token, wrong HTTP method, or incorrect JSON body would make the UI look broken even if the components render correctly. Testing api.js keeps this contract visible and makes backend/frontend

3.6.4.4 Manual UI Testing

Manual browser testing was used to confirm flows that are easier to judge visually than through isolated tests. The tested flows included signing in, invalid login feedback, switching between dashboard pages, checking role-based navigation, viewing sensor readings, selecting devices, opening room management, viewing charts, and logging out.

Responsive behavior was also checked with automated Playwright browser tests. The Playwright configuration runs the same black-box scenarios in desktop Chromium and a Pixel 5 mobile viewport. The responsive test verifies that the main dashboard and navigation remain visible and that the page does not create horizontal overflow.

Black-box Test Cases

Test ID	Test area	Test objective	Preconditions	Test steps	Test data / input	Expected result	Actual result	Automation	Result	Evidence
BB-01	Login	Verify that a user can log in with valid credentials.	Frontend is running, Backend login endpoint is available or mocked.	1. Open login page. 2. Enter username. 3. Enter password. 4. Click Sign in.	Username: `ADMINTEST`, password: `123456`	User is authenticated and redirected to the dashboard.	Dashboard opens on Home Overview with System Admin role.	Automated + manual	Passed	Playwright test `renders dashboard data after sign in` + manual browser test
BB-02	Login validation	Verify that invalid or missing login input is rejected.	Login page is open.	1. Submit empty fields or invalid credentials. 2. Check the login screen.	Empty fields; username `bad-user-manual-test`, password `wrong-password`	User stays on login page and sees validation or error feedback.	Empty fields stay on login form with required inputs invalid. Invalid credentials show `Invalid username or password.`	Manual	Passed	Manual browser test
BB-03	Role-based access	Verify that users only see features allowed for their role.	Test users or mocked login responses exist for `resident`, `building-administrator`, and `admin`.	1. Log in with each role. 2. Compare visible navigation pages and actions.	Roles: `resident`, `building-administrator`, `admin`	Each role only sees allowed dashboard views and controls.	Admin sees `Users`, `Devices`, and `Logs`. Building Admin does not see system-admin views. Resident sees only `Home`, `Sensors`, `Samples`, `Payload`, and `Alarm`; device filter is disabled.	Automated + manual	Passed	Vitest access-control tests; manual browser test with `ADMINTEST`, `MANAGERTEST`, and `Piotr`
BB-04	Dashboard display	Verify that sensor readings are shown correctly.	User is logged in. Readings API returns sensor data.	1. Open dashboard. 2. Inspect overview cards and latest reading values.	Sensor `101`, classification `Normal`, CO2 value from live backend	Dashboard cards display correct sensor values without layout issues.	Home dashboard shows `Device 101`, classification `Normal`, and live sensor values.	Automated + manual	Passed	Playwright test `renders dashboard data after sign in` + manual browser test

Figure 28 – Frontend black-box test case documentation covering login, role-based access, dashboard display, error handling, and responsiveness.

3.6.4.5 Test Results and Limitations

The latest local frontend verification was run on 2026-05-26. Vitest passed with 7 test files and 36 tests. Playwright passed with 6 browser tests across the desktop and mobile Chromium projects. The production build also passed with npm run build.

The automated coverage includes access-control logic, login UI, role selection, user menu logout, restricted notices, alarm controls, API request construction, API error handling, mocked login/dashboard black-box flow, backend-offline behavior, and responsive layout checks. This gives reasonable regression coverage for the prototype frontend.

The main limitation is that not every visual and data scenario is automated. Some dashboard navigation, chart readability, live backend data, and abnormal alarm states were checked manually or through mocked data rather than through a full end-to-end system test with real IoT readings. Another limitation is CI: the repository builds the frontend Docker image in GitHub Actions, but the Vitest and Playwright frontend test commands are not yet configured as a required CI job.

4 Results and Discussion

Author: Waqar Ahmed Khan, Piotr Gała

4.1 Final System Results

The final **AeroSense** prototype achieved an integrated monitoring flow from sensor readings to user-visible results and device alarm feedback. The IoT device collects environmental measurements and publishes them through **MQTT**. The backend receives the readings, stores them in MySQL and forwards the relevant values to the **ML service**, which returns a prediction of Normal, Cooking or Fire together with confidence and risk information. The frontend then presents the latest readings, device status, alerts and historical trends.

The system also demonstrates a response loop rather than only passive monitoring. When the model predicts **Fire**, the backend can publish a critical MQTT alarm message to the device so that its buzzer is activated. Keeping Cooking as its own prediction category supports the project goal of reducing unnecessary fire alarms from ordinary indoor activity.

Black-box testing on the containerised system verified the main integration endpoint with Normal, Cooking and Fire sensor profiles. The tested requests reached the backend, received ML predictions and were confirmed as stored in the database. The backend test suite also covers endpoint processing, ML communication, streaming and alarm mapping. This shows that the prototype works across subsystem boundaries, although it remains a decision-support prototype rather than a certified fire alarm installation.

4.2 Requirement Coverage

Several important requirements were **fulfilled** within prototype scope. The project uses GitHub as a central repository, provides automated testing through GitHub Actions and deploys its main services through Docker containers. The implemented system also supports the central functional journey: receiving sensor readings, making ML-based classifications, displaying monitoring information and returning an alarm signal to the IoT device.

Some requirements were **partly fulfilled**. The deployment configuration shows a working VPS-based hosted system, although the repository does not clearly identify the required public cloud provider. The frontend includes responsive layouts and role-based views, while the backend protects key endpoints using JWT authentication and role-based authorization policies. The ML model achieved strong evaluation results on the prepared data, but these results do not replace validation in real building conditions over a longer period.

Other requirements were **not completed or not evidenced** in the final implementation, especially around production-level security and infrastructure. The backend implements JWT authentication, but MQTT communication does not show visible TLS encryption, device-level certificate authentication is not implemented, and no serverless backend workload is present. These limitations are significant for a safety-related real-world system and should be treated as future development work rather than hidden by the successful prototype demonstration.

4.3 Subsystem Integration

Integration was a strong result of the project because the four technical areas were connected into one usable workflow. The IoT firmware publishes readings on the `iot/readings` MQTT topic. The MQTT-to-HTTP bridge sends them to the backend `POST /api/readings` endpoint, where the backend stores the input, requests an ML prediction and stores the resulting classification. The frontend retrieves readings and device information through REST endpoints and is designed to receive new readings through a Server-Sent Events live stream, while the backend can publish alarm commands on a device-specific MQTT topic.

The ML integration was kept consistent with the deployed system by using the shared measurements available at runtime: **temperature**, **humidity**, **TVOC** and **eCO2**. This avoided a mismatch between training inputs and live prediction inputs. The three-class model also provides a meaningful connection to the user problem because a Cooking event can be displayed without automatically being treated as Fire.

The completed implementation differs from parts of the early interface contract. That document proposed gRPC communication towards the frontend and a dual-rate sensor reporting strategy. In the delivered prototype, the dashboard uses REST with **Server-Sent Events** for live updates, while the firmware reports at a fixed interval. This trade-off simplified browser integration and made an end-to-end demonstration achievable, but it should be documented as an implementation change from the original design.

4.4 DevOps Reflection

DevOps practices helped the teams combine separate components with less uncertainty. The current **GitHub Actions** workflow builds and tests the .NET backend, checks the Python ML service with linting and coverage, builds and tests the IoT firmware using PlatformIO, builds and tests the frontend with Vitest and Playwright, and builds Docker images for the backend, ML service and frontend. The black-box endpoint tests are now merged into the `Dev` branch, providing documented evidence of the running containerised integration.

Deployment is supported through **Docker Compose** on a VPS. The deployed service definition brings together the MQTT broker, MQTT-to-HTTP bridge, MainServer, MySQL database, ML server and frontend. Health checks and service dependencies help the backend wait for required services before serving the integrated flow. The successful GitHub Actions run captured for the report is relevant evidence that build, test, image and deployment activities were performed through the shared workflow.

Automation still has limitations. Model retraining, approval and promotion are not automatically performed by the deployment workflow. A drift-monitoring script has been prepared for ML analysis, but it is not scheduled as part of the live service or CI/CD pipeline. In addition, the deployment job depends on the backend, ML, frontend and Docker jobs, but does not explicitly require the IoT firmware job to pass. A later production pipeline should tighten this gate and include stronger operational monitoring.

4.5 Challenges and Trade-offs

A major challenge was aligning embedded firmware, backend services, machine learning and frontend development around the same data contract. Sensor fields, classifications, identifiers and alarm messages needed to be understood consistently by each team. End-to-end tests were especially valuable because a component can operate correctly in isolation while still failing when connected to another subsystem.

The team also had to balance model quality with available data and project time. The ML dataset combines IoT and external smoke-sensor observations and is unevenly distributed between Normal, Cooking and Fire. This supported a useful prototype model, but performance measured on the prepared dataset cannot be interpreted as guaranteed reliability in every real apartment or sensor placement.

Finally, the team prioritised an integrated demonstration over production hardening. Container deployment, MQTT messaging and REST/event-stream integration made a working prototype possible within the semester. The trade-off is that encrypted device communication, device-level authentication, serverless infrastructure and broader real-environment validation were not completed. Being explicit about these gaps gives a more accurate result: AeroSense demonstrates a promising integrated prototype, with clear work remaining before any safety-critical deployment.

5 Conclusions

5.1 Project Conclusion

AeroSense demonstrates that IoT-based fire risk and air quality monitoring can be built as a working integrated prototype within a semester-long project. The delivered system covers the full data pipeline: an ATmega2560 IoT node collects environmental readings from temperature, humidity, CO_2, TVOC, and electrochemical sensors and publishes them every five seconds over MQTT. A central backend server receives and stores the data, then forwards each payload to a machine learning inference service that classifies room conditions as Normal, Cooking, or Fire using a trained Random Forest model. When a Fire result is returned, the backend publishes a CRITICAL alarm back over MQTT, which triggers the IoT device buzzer in real time. A React-based web dashboard gives residents, building administrators, and system administrators role-appropriate views of live readings, historical data, and alert status.

The system was successfully deployed as a containerised stack on Google Cloud Platform, with all components orchestrated through Docker Compose. A shared GitHub Actions CI/CD pipeline automates building, testing, and deployment for the backend, ML service, frontend, and IoT firmware. End-to-end integration was demonstrated with the full sensor-to-alarm path working as intended across all four subsystems simultaneously.

The project also surfaced clear limitations that would need to be addressed before any real-world deployment. The firmware and broker do not use encrypted communication, the backend does not enforce JWT authentication, and the ML model has been evaluated on prepared data rather than validated in a live building environment. These gaps are documented honestly in the future work section. AeroSense stands as a successful proof-of-concept that shows the approach is viable, while being transparent about what remains.

5.2 Learning Outcome

Working across four subsystems highlighted the importance of agreeing on a shared data contract early. The MQTT topic structure, JSON payload format, and classification labels became the central reference point that all teams worked against. Changes to any of these required coordinated updates across the full stack, which reinforced the value of maintaining a clear interface specification rather than relying on informal communication between teams.

The project also showed that automated testing and CI/CD pipelines bring real benefits even at prototype scale. Having the pipeline run firmware builds, unit tests, and ML model evaluations on every push caught integration issues earlier than manual testing would have. Building cross-team end-to-end tests demonstrated that components can behave correctly in isolation while still failing when connected, which confirmed that integration testing is essential rather than optional in a multi-team project.

From a machine learning perspective, the team learned that accuracy measured on a prepared dataset does not directly translate to confidence in deployment. The dataset combined sources recorded under different conditions, and the class imbalance between Normal, Cooking, and Fire required careful handling during training and evaluation. This experience highlighted that a model which performs well in testing still needs real-environment validation before it can be trusted in a safety-related application.

Finally, the semester demonstrated the practical challenges of synchronising four independently developed subsystems around a single running system. Regular integration sessions, a shared MQTT broker, and documented interface contracts reduced the number of surprises during final integration. For future projects of similar scope, establishing these coordination mechanisms at the start of the project rather than mid-way would save significant debugging time.

6 Future Work

Author: Soma Nagy

AeroSense was developed as a prototype system within the constraints of a fourth-semester project. While the current implementation demonstrates the core concept of IoT-based fire risk and air quality monitoring, several areas have been identified where the system could be extended and improved in future development cycles. This section describes the most important technical, user experience, DevOps, and long-term improvements that could be addressed to move AeroSense from a working prototype toward a more complete and production-ready solution.

6.1 Technical Improvements

The current IoT node relies on a fixed set of sensors including temperature, humidity, CO_2, TVOC, and a smoke or air-quality indicator. One important technical improvement would be to add dedicated gas sensors capable of distinguishing specific combustion gases such as carbon monoxide and methane. This would allow the system to make more precise classifications between real fire events, cooking activity, and false alarm situations. Expanding the sensor suite would also provide richer data for the machine learning model, improving the accuracy of its predictions over time.

The machine learning model used in the current prototype was trained on a limited dataset. A significant improvement would be to retrain the model on data collected from real AeroSense deployments over a longer period. Collecting labeled sensor readings from different room types and real events would allow the model to generalize better and reduce the number of false positives and missed detections. Introducing online or incremental learning could further allow the model to adapt to the specific patterns of a given room or building over time.

The MQTT-based communication between the IoT device and the backend works well for the prototype but does not include end-to-end encryption or device-level certificate authentication. Future development should introduce TLS

encryption for the MQTT broker and implement a proper device certificate or token system so that only verified IoT nodes can publish sensor data to the backend. This would make the system more robust against spoofed data and unauthorized devices.

Local alarm logic on the IoT device currently triggers the buzzer based on a simple threshold check. Improving this logic to also consider trend-based or rate-of-change conditions would reduce unnecessary buzzer activations. For instance, a rapid rise in CO_2 combined with a temperature increase could be treated differently from a slow gradual increase, allowing the system to distinguish between an emerging problem and normal environmental changes.

6.2 User Experience Improvements

The current dashboard provides the core functionality needed to view room conditions, alerts, and historical data. However, the layout is primarily designed for desktop use. Improving mobile responsiveness so that residents and building administrators can comfortably use the dashboard on phones and tablets would significantly increase the practical value of the system, especially in situations where a quick check is needed away from a desk.
Warning and alert messages in the current system use a standard notification format. A future improvement would be to make warnings more contextual and informative, for example by explaining not only that a risk has been detected but also which sensor values triggered the warning and what the recommended action is. Clearer and more specific messages would help residents understand the situation without needing to navigate to a detailed data view.

Notification preferences are currently uniform for all users. Adding configurable thresholds and notification channels per user would allow residents and building administrators to decide which events they want to be alerted about and how, for example through email, push notifications, or SMS. This would reduce notification fatigue and allow different users to adjust the system to their own monitoring needs and working patterns.

6.3 DevOps and Deployment Improvements

The current CI/CD pipeline handles automated building and testing for the backend, frontend, and ML service. A meaningful improvement would be to expand the test coverage in the pipeline, particularly by adding integration tests that verify the full data flow from MQTT ingestion through backend processing to the frontend display. This would help catch regressions that involve interactions between subsystems, which are not covered by unit tests alone.
The prototype is currently deployed on a VPS without a separate staging environment. Introducing a staging environment that mirrors the production setup would allow the team to verify new releases in a realistic context before making them available. This would reduce the risk of deploying broken changes to the live system and provide a safe environment for testing configuration changes, database migrations, and new features.

Adding runtime monitoring and centralized logging would improve the ability to detect and diagnose problems in a deployed system. Tools such as Prometheus and Grafana could be used to track service health, response times, and system resource usage. Centralized log aggregation would make it easier to investigate failures across multiple services without needing direct server access, which is especially important as the number of connected rooms and devices grows.

6.4 Long-term Product Ideas

The current system is designed around monitoring individual rooms within a single building. Extending the architecture to support multiple buildings from a single backend deployment would make AeroSense viable for property management companies, schools, or larger facilities. This would require changes to the data model, access control, and dashboard to allow building administrators to navigate between multiple buildings and get an aggregated risk view across locations.

In a real deployment scenario, AeroSense could be integrated with emergency response infrastructure. For example, when the system detects a confirmed fire risk above a certain confidence threshold, it could automatically notify a building security system, trigger a centralized alarm panel, or send an alert to a facility management platform. This kind of integration would require standardized APIs and careful design to avoid false alarms triggering emergency responses, but it would significantly increase the real-world value of the system.

The historical data collected by AeroSense over time could be used for stronger analytics beyond the current room condition classification and short-term predictions. Long-term trend analysis could identify rooms that consistently show elevated CO_2 levels during certain hours, or buildings where temperature and humidity patterns suggest poor ventilation. These insights could support building management decisions such as scheduling ventilation maintenance, identifying high-risk areas before an incident occurs, or evaluating the impact of physical changes to a room.

Finally, a data export feature would allow users and administrators to download historical sensor readings, alerts, and room condition reports in a structured format such as CSV or PDF. This would support external audits, incident reporting, and integration with third-party building management software. It would also give building administrators a convenient way to document environmental conditions over time for regulatory or insurance purposes.

VIA University College

AeroSense- Process Report

Semester Project 4

Group Y1

Students

Felipe Figueiredo (355463)

Guillermo Sánchez (355442)

Waqar Ahmed Khan (355425)

Eliza Manciu (204590)

Piotr Gała (355451)

Matteo Saccucci (355400)

Matteo De Filippis (355438)

Soma Nagy (355465)

Supervisors

Marketa Tranberg

Joseph Chukwudi Okika

Kasper Knop Rasmussen

Erland Ketil Larsen

Character Count: 45424

Word Count: 7425

Software Technology

Engineering 4th Semester

May 2026

Contents

1 Introduction

The AeroSense project was initiated with the goal of creating an intelligent fire-risk and environmental monitoring prototype for building residents. The project focused on developing an integrated system in a team environment, combining IoT sensors, backend services, machine learning and a frontend dashboard to support clearer monitoring and reduce unnecessary alarms.

2 Group Work

Many of our group processes began with forming a larger team whose members had different technical interests, experiences and working styles. To establish a shared foundation, we created a group contract covering communication, deadlines, decision-making, workload distribution and conflict resolution. Since AeroSense was divided into IoT, Backend, ML and Frontend responsibilities, clear task ownership was necessary, while regular coordination was important whenever one subgroup's decisions affected another.

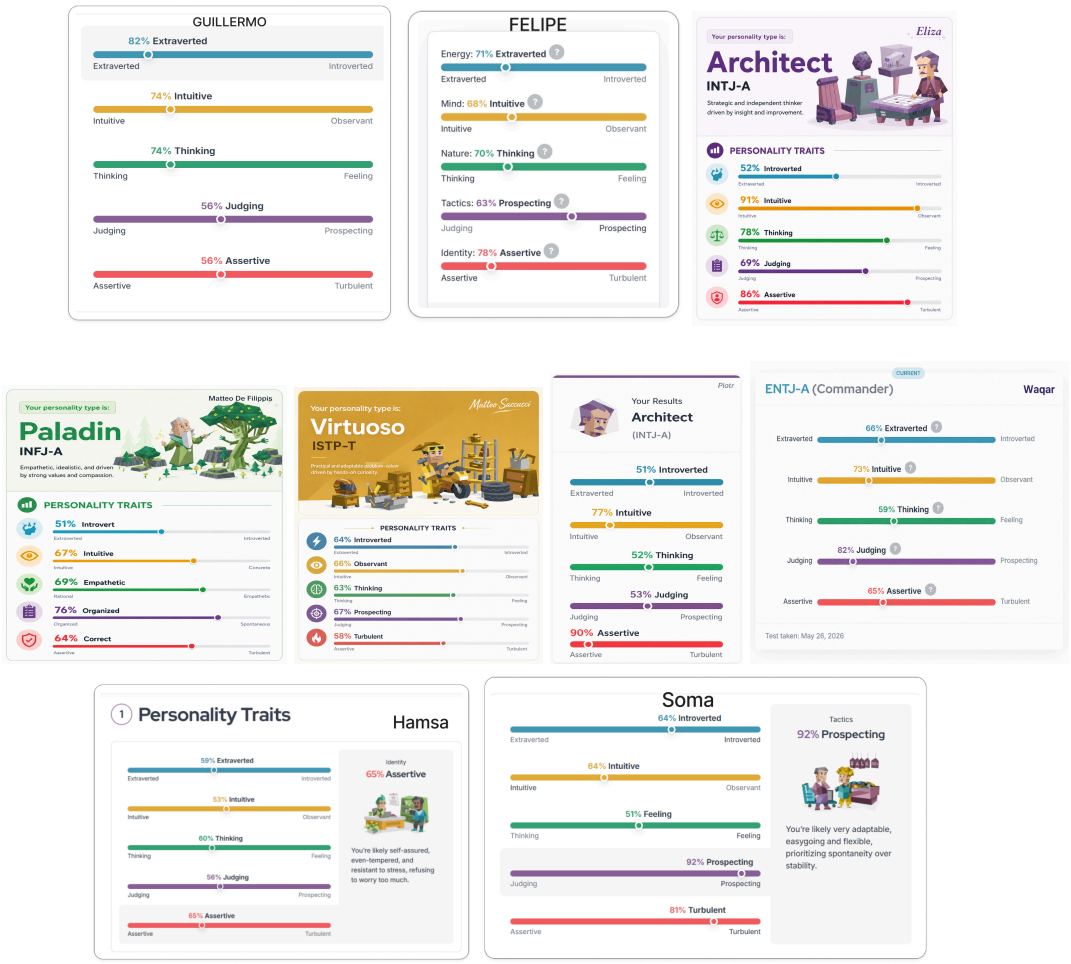

Figure 1: Personal Profiles

Our team was culturally diverse as we come from: Italy, Spain, Brazil, Pakistan, Romania, Hungary, Poland, Slovakia. The Southern European members of our team exhibited flexible relationship-based communication which proved to be a good balance for the structured task-oriented methods used by Central and Eastern European members. Having no experience working together earlier made us be skeptical at first about each other. But later on we got to know each other little by little and at the end we managed to work in a proper way. At the end of the day, we have learnt that with proper communication and will to

understand and hear different ideas we can work together.

We based the group contract for the project on our previous semesters SEP groups contracts with slight changes, for example to the point system. This group contract outlined our core values, purpose, expectations, conflict resolution strategies, and accountability measures in the team. While social loafing still occurred in the group, which we considered normal given the size of it, the team tried to handle it by having a strict contract with strict penalties. We started going by the contract but after some team members were not content with it (even though they had signed it) we decided to not apply any of the penalties and just have talks whenever a problem arised.

2.0.1 Professional Collaboration

To document our ability to independently take part in professional collaboration, we adopted industry-standard practices such as using git and Jira. We used branching with Pull Requests to review code, ensuring no single person could break the main build. We communicated asynchronously via Discord to respect deep work time.

An important feature of the project's process was the role of a Product Owner (PO). The PO was responsible for managing the user stories, prioritizing the backlog, and ensuring that the team was aligned with the project goals. This role was crucial in maintaining a clear vision and direction for the project.

For the best of the project, we decided to maintain the roles from the beginning, providing consistency and a clear point of contact for the team regarding project requirements and priorities. The role of the PO was assigned democratically at the start of the project by votation.

3 Project Initiation

The initial idea for AeroSense developed from a real problem at Kamtjatka Student Residence: fire alarms can be triggered by everyday activities such as cooking or steam, creating unnecessary disruption and reducing trust in alarms. The team selected this problem because it had clear practical relevance and could be addressed through the combined skills of the IoT, Backend, ML and Frontend sub-groups.

Aligning on the actual vision was more complicated than identifying the problem. We had to agree on whether AeroSense should focus mainly on air-quality monitoring, fire-risk prediction or false-alarm reduction. This required several discussions about the target users, sensor data, alert behaviour and what the final prototype should demonstrate.

One challenge during initiation was defining a realistic project scope. It was easy to think of AeroSense as a complete safety product for a residence, while the semester allowed only a prototype. We resolved this by defining the system as a monitoring and decision-support solution rather than a replacement for certified fire alarms, while still allowing future improvements.

Another challenge was approaching the problem in a data-driven and responsible way. Because real fire testing was unsafe and impractical, the ML team needed external smoke-sensor data together with available IoT readings. This helped the project evaluate Normal, Cooking and Fire situations without making unsupported assumptions about real-life safety performance.

3.1 Choice and Alignment on Framework

Shortly after the group formation, the team agreed to use Agile Unified Process (AUP) together with Scrum. AUP gave the project its overall phases: Inception, Elaboration, Construction and Transition, while Scrum supported sprint planning, regular meetings and incremental development. Because AeroSense involved several technical areas, the team was divided into IoT, Backend, ML and Frontend responsibilities, with coordination needed across the sub-groups.

4 Project Execution

The way of working in the project required more coordination than projects focused on a single application. We adopted Agile practices that fitted the development of an integrated prototype. These included sprints, weekly meetings, task assignment, GitHub branches and pull requests, a shared task board, and continuous testing through the CI/CD pipeline.

Our intended workflow was initially described in the project description, time schedule, risk assessment and group contract. These documents helped us agree on responsibilities, communication expectations and the planned use of Scrum and AUP. However, the actual project required adjustments, especially when the different subsystems began to depend on each other during integration.

An important part of our work was allowing the sub-groups to progress independently while still coordinating shared decisions. For example, the ML team could work on preprocessing and model evaluation separately, but deployment depended on alignment with the backend API and the IoT sensor fields. This balance between asynchronous work and regular integration discussions helped the team make progress while still producing one connected AeroSense system.

4.1 Before Project Period

Because of daily school and work commitments, we mostly worked asynchronously in this period. Our sprints were one week long, starting and ending on Tuesdays. Each sprint would start around lunchtime as we wanted to have time to end the previous sprint collaboratively. Each sprint would start with a planning meeting where we would discuss the goals and tasks for the sprint. At the end of each sprint, we would review all features and tasks, and discuss any blockers or challenges faced during the sprint.

4.2 Project Period

Our work during the project period looked as follows:

4.2.1 Week perspective

We met every working day, weekends voluntarily individually except the last weekend before submission, which was mandatory.

4.2.2 Day perspective

We preferred to work remotely as this reduced commuting time, allowed for more flexible working hours, but also prepared us for future remote work.

Each day started at 9 with a daily standup meeting where we discussed for each: - What was done - Any blockers - What will be done - Questions

After the standup, we would plan the next two day's work, which we considered one sprint during this period. Based on our framework, the purpose and focus of a meeting is to merely label and agree on which tasks we add to the next sprint on Jira to focus on during the sprint.

Before the project period, all the members of the group met at least once a week. Once the project period started, we had short group meetings every morning at 9, and after each meeting we divided into sub-teams to work collaboratively, this helped us be more efficient and solve relevant questions within the sub-teams.

Each day the meetings lasted approximately one hour, and later we had active communication through discord throughout the day. Depending on everyone’s situation, each of us was able to freely choose their schedule and work whenever it was more comfortable according to their schedules and preferences.

4.2.3 Feature perspective

Working on a feature was usually for 1-3 people within sub-teams. The feature was started by discussing

the definition of done, breaking down the tasks step by step, and creating a branch for it. Each feature was practically a vertical slice of the product and the work was also often split into vertical slices in order to reduce interpersonal and sub-teams dependencies. The aim of each feature team was to deliver a working feature compatible with main (which was evolving in parallel) once the feature was done. Each feature would have to be reviewed by someone else in the sub-team before merging to main by using pull requests.

In order to be more efficient, the following technologies were implemented in the project, we used GitHub for version control, Jira for Scrum organization and task distribution, Figma/FigJam for brainstorming and unrestricted diagramming, PlantUML inside VSCode for quick UML sketches, and Discord for screen sharing and communication.

4.3 Adherence to the Framework

It must be noted that the adherence was not perfect. At first Figma was used for all the work, some members were sceptical because they had never used it but they gave it an opportunity. After a few iterations the group decided to change to the standard industry solution, Jira, so we migrated our Sprints. In the figure below we can see an example of an iteration designed in Jira:

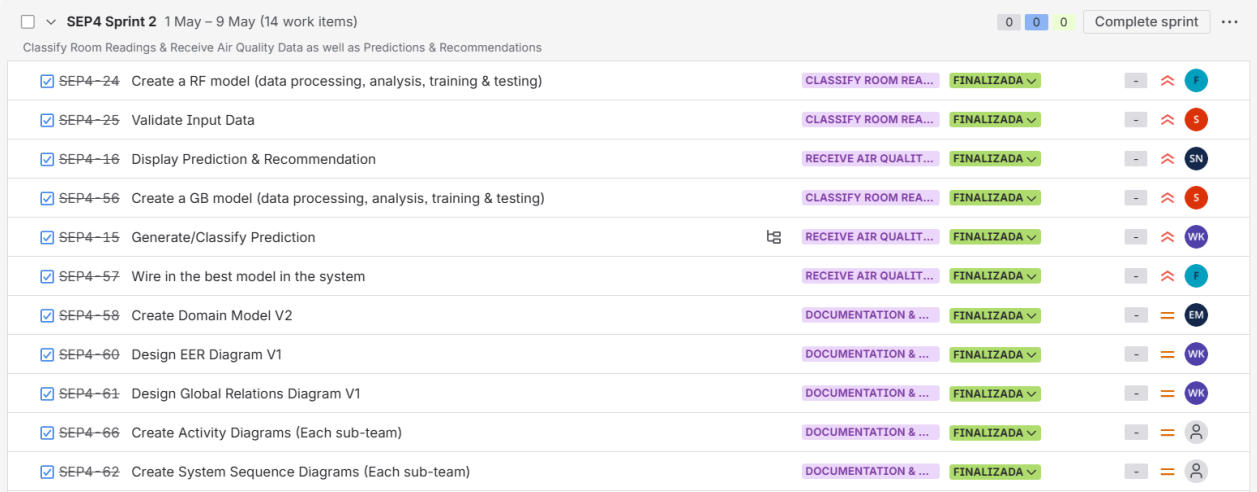

Figure 2: Sprint's task organization

Additionally to the group iterations, each sub-team had the freedom to also divide tasks within sub-teams in order to achieve a more narrowed perspective of the sprints, in the figure below is an example of how this sub-sprints look:

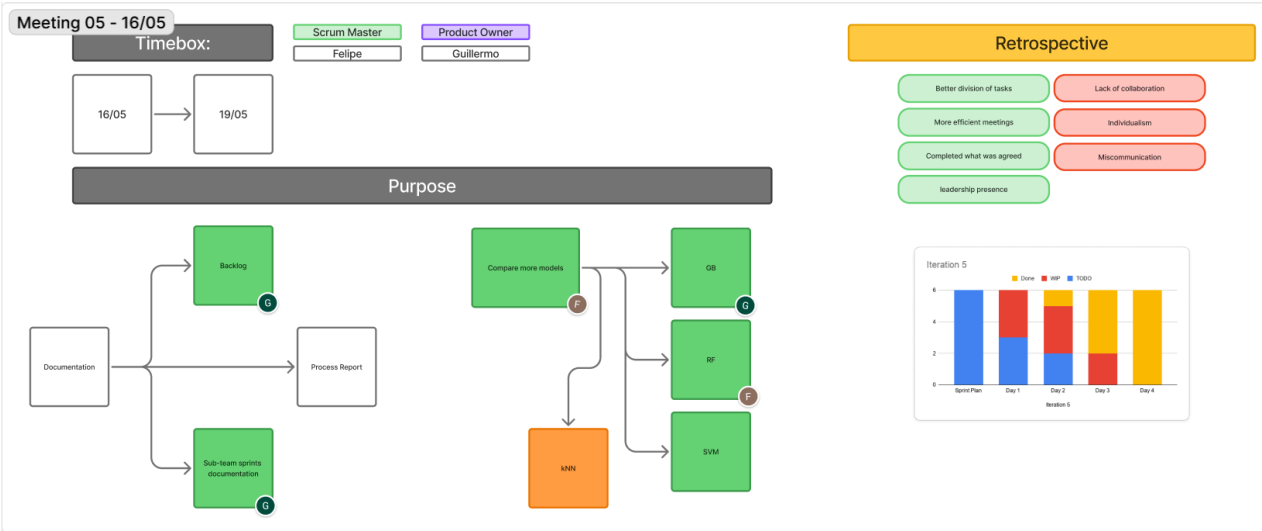

Figure 3: Sub-team Scrum implementation

5 Choice of Tools and Methods

5.1 Task Decision and the PO Role

In order to facilitate the product owner to choose the next task a backlog was created to keep track of the features that were already working, their priority for the project and their estimated effort. In practice, the backlog helped to have a cohesive perspective of the project and the features we would like to ideally have implemented. A snapshot of the backlog can be seen below:

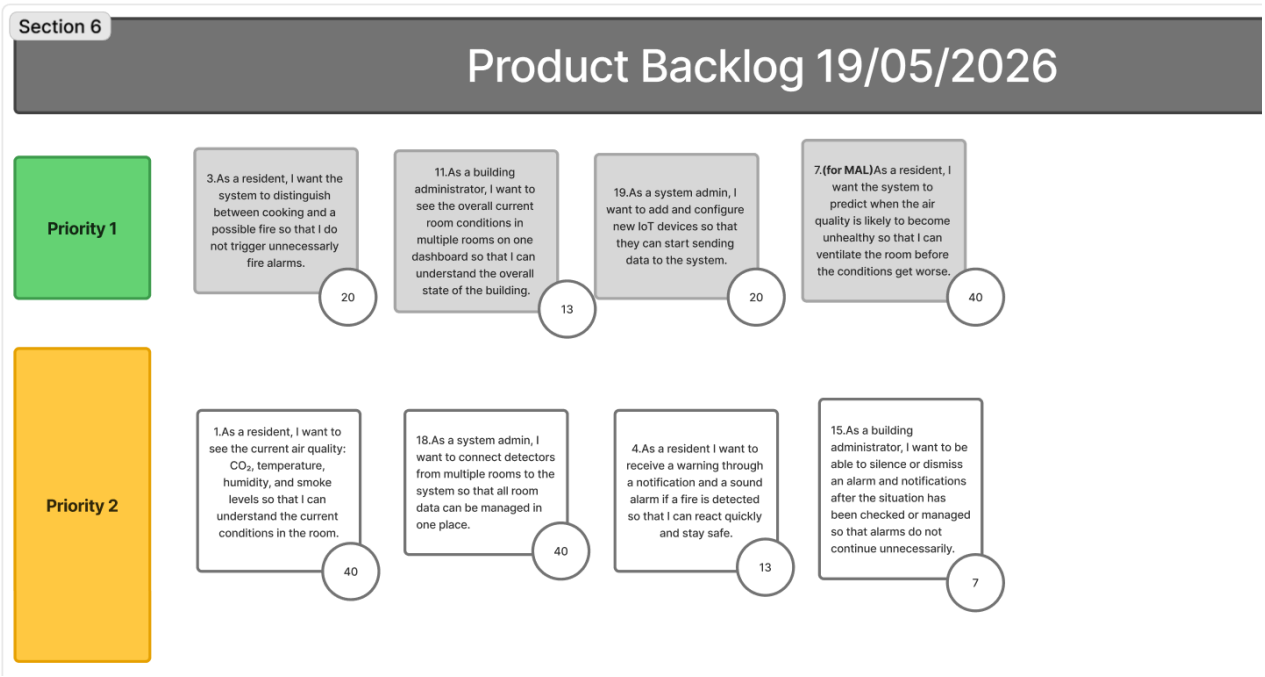

Figure 4: Product Backlog

Creating concepts from sources like user stories, non-functional requirements and others also proved to be a powerful idea that arised from the inception phase. This method allowed great flexibility while

keeping the opinionated choices of prioritization - often arising from the data-driven nature of the project.

5.2 Tools

The most used development tool by all members was Visual Studio Code by Microsoft, since it provided a shared coding environment across the different sub-teams. For documentation, we initially used Markdown with Pandoc, and later switched to Google Docs for a better live collaboration experience. Markdown allowed us to collaborate with fewer merge conflicts while keeping the focus on content creation rather than styling. Google Docs was useful for group writing sessions because all members were already familiar with it.

PlantUML was chosen as the standard tool for diagrams. It allowed quick diagramming and sketching of ideas directly from the development environment, which made it easier to update diagrams as the project evolved.

Figma was also used to keep track of user stories, use case diagrams, use case descriptions, database design, and sub-team iteration management. It proved especially useful for brainstorming and quick collaboration during the inception phase.

PlatformIO was used by the IoT sub-team for coding, testing, building, and uploading code to the device. This helped organize the embedded code and accelerated the development process.

For the frontend sub-team, React was used to build the dashboard, with Vite as the development and build tool. ESLint, Vitest, React Testing Library, and Playwright were used to support code quality, unit/component testing, and end-to-end testing of the frontend flow.

Visual Studio Code also supported efficient database work by allowing the team to connect to the database and execute queries from the same environment used for development.

GitHub was used for version control, branching, pull requests, and collaboration. GitHub Actions supported the CI/CD workflow by running automated checks such as tests, builds, and deployment-related steps during development and integration.

Discord was used as the main communication tool for online meetings, sub-team chats, quick decisions, screen sharing, and sharing links or updates during the project.

6 Personal Reflections

6.1 Guillermo Sánchez Martínez

At the start of the project I was a bit skeptical about how well we could collaborate given the size of the group. There was also some uncertainty since I had never worked with most of the group members, so at the beginning I felt I was being very strict with the other team members, probably too much. And I believe they got a wrong impression of me.

Once we started having meetings and working together I started to understand the members of the team and how I could help in order to achieve a great project performance. The meetings went better than I expected and most members of the group surprised me for the good.

In the project period everyone had already adopted the role they are most comfortable with. Some liked to have long discussions in discord and were very proactive, others had small talks and others did not like to participate actively. At first I thought this last group was going to be problematic because their lack of proactiveness made me think that they were not working, but I was wrong. During the sprints this last group proved to be working as everyone else, they were just more quiet.

About my performance, I liked being an active member of the group, both in the big group and in the sub-team. I personally enjoy having a role where I not only have to worry about my tasks, but also about my teammates performance and task distribution. In this project I narrowed my responsibilities to ML. So I

6.3 Hamsa

The SEP4 AeroSense project ran across a large team split into sub-groups covering IoT, backend, frontend, and ML. The goal was concrete: fewer false fire alarms through smarter environmental monitoring and classification.

I spent most of my time in the IoT layer, which turned out to be the hardest part to stabilize. Embedded hardware is unforgiving. WiFi dropped unexpectedly, MQTT messages failed silently, sensor readings came back unreliable. Fixing those issues required patience and methodical debugging. It helped that the risk assessment had already flagged cloud-sensor communication as a critical risk, so we weren't surprised when it broke.

My personality type is ENTJ-A. In practice, that meant I naturally gravitated toward keeping things moving: spotting blockers early, coordinating between teams, pushing for decisions when things stalled. During the Construction phase, when deadlines tightened and integration pressure rose, that worked in my favor. It also exposed a blind spot. I had to actively slow down and listen more, especially when teammates had a different read on a problem.

Working with Scrum and Kanban across multiple sub-teams was new at this scale. I got genuinely more comfortable with branching strategies, pull requests, and debugging issues that spanned layers I didn't own. With this many moving parts, shared visibility on GitHub mattered more than I expected.

I leave this project more confident debugging distributed systems, and clearer about how I work best: structure, defined ownership, and enough room to be wrong.

6.4 Piotr Gala

During this project, I worked mainly as part of the Frontend team. My work included improving the dashboard, displaying sensor data and alerts more clearly, adjusting role-based access and views, fixing parts of the JWT-based flow, and adding frontend tests. I also contributed to the frontend CI workflow, which helped make the React part of the project easier to verify before merging changes.

From a technical perspective, I think the frontend turned out quite good. The dashboard became clearer and more usable during the project, especially for showing sensor readings, alerts, and role-based information. I also worked with frontend testing and CI, which made the frontend easier to verify before merging changes. A lot of frontend work depended on backend data, authentication, user roles, and available sensors, so I also got more experience with building UI around data that was not always stable or complete yet.

The most difficult part of the project was not the implementation itself, but the process around it. At the beginning of the semester we worked more as one large group, and only later the subteams became more clearly separated. Because of that, communication problems were visible both between individual people and later between the technical subteams. Tasks were not always clearly defined, expectations were sometimes unclear, and it was not always obvious who was responsible for finishing or unblocking a feature. When the frontend started depending more on backend data, authentication, user roles, and sensor availability, these communication issues became even more visible.

The project also showed me that a large group needs stronger structure than a small one. Even if individual people are technically capable, the whole project can still become inefficient if communication, ownership, and expectations are unclear. In the beginning, I tried to be more engaged in the group process, but over time I realized that I cannot take responsibility for other people’s motivation or seriousness towards the project. Everyone in the group was responsible for their own work, and I did not want to spend too much energy trying to push adults to care about tasks they should already care about. Looking back, I still think this was a reasonable boundary, but I could have handled my own side better by communicating blockers earlier and not postponing my own work when the process felt messy.

The main thing I learned is that I work best when tasks are concrete, ownership is clear, and dependencies are visible early. For the next project, I would try to define frontend tasks more clearly from the start, push for earlier integration between teams, and avoid delaying my own work even when the group process is frustrating. This project taught me less about a perfect way to work and more about how a team should not operate if it wants to stay predictable under deadline pressure.

6.5 Waqar Ahmed Khan

At the beginning of SEP4, I felt excited and well prepared for the semester. I had developed stronger technical skills from previous semesters, and I chose to work with Machine Learning because I wanted to challenge myself in an area that was both interesting and important for the AeroSense project. I saw ML as a central part of the system because it connected the sensor data to the project's main purpose: distinguishing between normal conditions, cooking activity and possible fire situations.

However, one of the biggest challenges I experienced was communication within the ML sub-team. During the early part of the project, especially up to the proof-of-concept sprint, I often tried to arrange meetings so we could decide together what we were going to build and how we would divide the work. My intention was that we should work step by step as a team, contribute fairly and maintain a shared understanding of the solution. In practice, this was difficult. Messages were not always answered, and I often had to approach team members physically after class to understand what had been done or what the plan was.

This affected my motivation because I wanted the collaboration to feel professional and equal. One example was during the proof-of-concept work, where we divided tasks so I would work on the database part of the Main Server while another member worked on domain logic. After finishing my part, I wanted us to meet, merge the code and test it together end to end. Instead, I found out that the database part had also been done separately because it was believed that two people working on it would make the solution harder to integrate. From my perspective, this made my effort feel wasted and I felt disrespected, not because someone else solved the problem, but because the decision was made without proper communication.

At one point, the situation became serious enough that I considered leaving the SEP4 project or moving to another group. I discussed the issue with a supervisor and received advice to clearly warn the ML team that communication and cooperation needed to improve. After speaking with the sub-team, they agreed to respond more consistently and work in a more coordinated way. This did improve the situation to some extent in the following sprint, although the same communication challenges did not disappear completely.

Another challenge was agreeing on the ML direction. At first, there were discussions about each member finding different datasets and training separate models. I was concerned about this because models trained on different datasets cannot be compared fairly. I believed we should use the same dataset so that model performance could be evaluated on a common basis. Later, the team agreed that using one shared dataset would make the comparison more meaningful, which I think was the right decision for the project.

There were also disagreements about model ownership. At one point, the team agreed that different members would work on different models. I was assigned KNN and SVM, while other members worked on Random Forest and Gradient Boosting. I accepted this at first, but later I realised that I was not contributing directly to what became the most important model choices for the final system. This frustrated me because I wanted my work to have a significant impact on the final ML solution. I communicated this to the team, and although it led to difficult discussions, I think it was important to speak honestly about contribution and responsibility.

One lesson I learned is that commit count alone does not always show a person's real contribution. In my case, some of my work was replaced, overwritten or changed because decisions were made without enough coordination. Also, some significant parts were completed by individual members before the rest of the sub-team had a chance to participate. This made it harder to create a fair and shared sense of